UNIT 9 INSTRUCTION SET ARCHITECTURE

9.0 Introduction
9.1 Objectives
9.2 Instruction Set Characteristics and Design Considerations
    9.2.1 Operand Data Types
    9.2.2 Types of Instructions
    9.2.3 Stored Program Organization
9.3 Number of Addresses and Instruction size
9.4 Instruction Set and Format Design Issues
9.5 Addressing Schemes
    9.5.1 Immediate Addressing
    9.5.2 Direct Addressing
    9.5.3 Indirect Addressing
    9.5.4 Register Addressing
    9.5.5 Register Indirect Addressing
    9.5.6 Relative Addressing Scheme
    9.5.7 Base Register Addressing
    9.5.8 Indexed Addressing Scheme
    9.5.9 Stack Addressing
9.6 Summary
9.7 Answers/Solutions

9.0 INTRODUCTION

In the previous two blocks, you have learnt the concepts of data representation, memory organization and Input/output organisation. This Unit discusses about one of the most fundamental aspect of a general-purpose computer – the instruction. A computer can do general purpose or specific tasks using the instructions. Instructions are the mediator between the programmer and hardware. Hardware is the computer architecture, and software is the instruction set architecture. Instructions set architecture is the only way you can interact with the computer machines. An instruction set architecture consists of a complete set of instruction to do a task on a specific computer system.

In this unit, details of instructions format, operands data types, instruction types and various addressing modes have been discussed.

9.1 OBJECTIVES

After going through this unit, you will be able to:

- Define various characteristics of instruction set of a computer
- Appreciate various components of instructions
- Explain the design of different instruction
- Define the term instruction format
- Explain various addressing modes used in an instruction.

9.2 INSTRUCTION SET CHARACTERISTICS AND DESIGN CONSIDERATIONS

A programmer writes the program instructions in assembly language, which are easily understandable for the humans. But the assembly language is not understandable by the computer machines, as these computer machines are made of digital modules, which understand only binary logic. Hence, during execution, program is converted into binary codes which is understandable by the computer machines. A binary coded instruction is in format, which is interpreted and executed by the machine. Instruction format is discussed next.

Instruction format

A simple instruction format is shown in Figure 9.1. It consists of three components: the operand address, the operation code or opcode and the mode. The number of bits is allocated to each component. Basic definition of these components are:

Opcode: In instruction, opcode decides the operation to be performed on the data.

Operand: In Instruction, operands are the data on which operation is to be performed.

Effective address: In instruction, it is the memory address where the data/operand is stored. For example, if data is stored in register, register is the effective address. Effective address is decided by the addressing schemes, which are discussed in details in section 9.5. The mode field of the instruction determines, how the effective address would be computed in a machine.



Figure 9.1: Instruction format components

The total number of operations, which can be performed by a computer machine is decided by the bits allocated for the operation code of an instruction. The n-bits operation code can be used to code 2^n distinct operations. For example, the 7 bits opcode, as given in Figure 9.1, can be used to code $2^7 = 128$ distinct operations. The computer can be designed using these opcodes, such that a typical opcode may represent, a typical operation. An example opcode can be designed as:

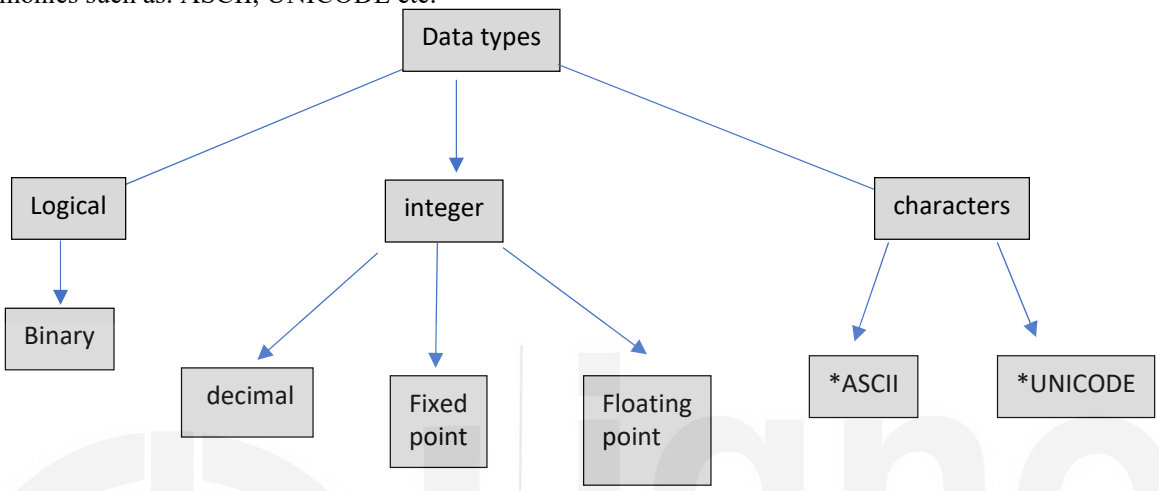
ADD operation: 1100100 (7-bit opcode)

The number of bits allocated to operand field, of the instruction in Figure 9.1, depends on the memory address size or the data bits. In Figure 9.1, the 31st bit "I" decides the mode if address is in direct or indirect mode. In direct mode, the address given in the instruction is the effective address of the operand. In indirect mode, the instruction holds the memory address where the effective address of operand is saved. It may be noted that this instruction format just allows direct and indirect addressing mode and a single operand in each instruction. However, different instruction formats may support different types of addressing modes and different number of operands.

A point that can be noted for the instruction format of Figure 9.1 is that size of operand filed of instruction is 24 bits. In case, this operand is a direct operand, then the size of the main memory addresses that can be supported by the machine having instruction format, as given in Figure 9.1 is $2^{24} = 16$ M memory words.

9.2.1 Operand Data Types

In computer, operands are the data bits on which operation is to be performed. As shown in ***Figure 9.1***, data types can be categorised as logical, integer and characters. Logical data is Boolean which may be 1/0 or true/false. Integer data may be further divided into: decimal, fixed point, and floating point. And the character data have mnemonics such as: ASCII, UNICODE etc.



*ASCII : **A**merican **S**tandard **C**ode For **I**nformation **I**nterchange

*UNICODE: **Universal Character Encoding**

Figure 9.1: operand data types

9.2.2 Types of Instructions

To evaluate any computable function, a computer must have a group of instructions created by the designer in machine language. The set of instructions include various types such as: arithmetic instructions, logical instructions, shift instructions, instructions to transfer content from memory to processor register, program control instructions and input/output instructions. Broadly, instruction can be categorized as below:

1. Instructions to transfer the data
2. Instruction to manipulate the data
3. Instructions for program control

Instructions to transfer the data: In computer machines data transfer takes place between processor registers, between memory and processor register, between processor register and input or output interface. The instructions with data in the central processor register are faster than that of instructions with data in memory. Each instruction has a mnemonic which can be used as part of the assembly language. Please note that these mnemonics may vary for different machines. Data transfer instructions are listed below:

Table 9.1: Some Data transfer instructions of computer

Instruction	*Mnemonics*	*Remarks*
Load	LD	To load data from memory to accumulator register
Store	ST	To store data from processor register to memory
Move	MOV	To transfer data between processor registers
Exchange	XCH	Exchange data between processor registers or between a register and memory
Input	IN	Data transfer between processor register and input terminal
Output	OUT	Data transfer between processor register and output terminal
Push	PUSH	Data transfer from register to memory stack
Pop	POP	Data transfer from memory stack to register

Instruction to manipulate the data: Data manipulation instructions are categorized based on the type of operation the perform on the data. The following are the basic categories of data manipulation instructions:

- Arithmetic
- Logical and bit manipulation
- Shift instructions

Arithmetic: These instructions perform arithmetic operations on the data. Various arithmetic operations are: addition, subtraction, multiplication, division etc.

Logical and bit manipulation: The logical and bit manipulation functions are used to perform logical operations or manipulation by setting/resetting a single data bit. Some of the logical operations are -AND, OR, XOR etc.

Bit manipulation - selective set and mask: Let's assume A= 1010, and you need to set the least significant bit (LSB), keeping all the remining bits unchanged. Since you just need to set the LSB, therefore, the second operand for selective set would be B=0001 and you will use OR operator, as shown below:

A	1010
B	0001
A OR B	1011

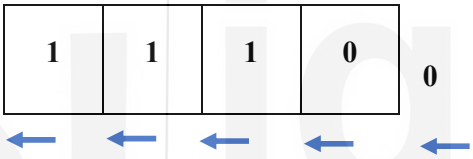
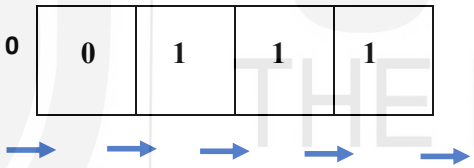
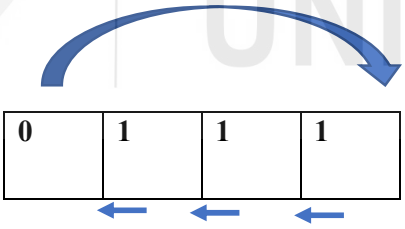
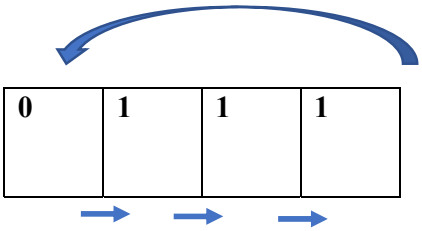
You may please notice that upper three bits in the result (A OR B) remains unchanged, whereas, the LSB is set to 1. Hence, least significant bit of A is set to 1 by performing the OR operation. Similarly, AND function can be used to clear the selective bit. For example, if for the given A value (1011), you just want to use the

LSB or in other words you would like to make the upper three bits as 0's. This is performed with the help of AND. For this example, the value of B would be selected as 0001. This operation is also called the mask operation and is shown below:

A	1011
B	0001
A AND B	0001

Shift instructions: Shift instructions are to shift the register data bits in left or right direction. In *shl R*, data bits of the register are shifted left where data bit input takes place at the rightmost bit and leftmost bit is lost. Whereas in *shr R*, data bits are shifted right and data is input from the leftmost bit. In *shr R*, rightmost bit is lost. In case of circular shift no bit is lost. In *cil R*, data bits move to the right and rightmost bits is stored in the leftmost bit. In *cir R*, data bits move to the left and leftmost bit is stored in the rightmost bit. The Table 9.2 shows these two types of shift operations.

Table 9.2: Shift instructions

Symbolic representation	Description	Functioning	Remarks
R ← shl R	Shift left register R		Left most bit is lost Answer is 1100
R ← shr R	Shift right register R		Right most bit is lost Answer is 0011
R ← cil R	Register circular shift left		Left most bit is moved to rightmost bit, no bit loss. Answer is 1110
R ← cir R	Register circular shift right		Right most bit is moved to left most bit, no bit loss Answer is 1011

Program control instructions: The address of the next executable instruction is stored in the program counter register. Program control instructions contains the condition, which may cause address alteration in the program counter. Hence, the execution of

program control instruction results into change in program counter address breaking the sequence.

Table 9.3 : Program control instructions

Instruction	Mnemonics	Function	Remarks
Branch	BR	To branch at particular address	conditional or unconditional
Jump	JMP	Jump to a specific address	unconditional
Skip	SKP	To skip the next instruction	conditional
Call	CALL	Call is used with subroutines	To call a function
Return	RET	To return back to sequence after function call	To return after executive a function

Branch (BR) and Jump (JMP): Branch and jump instructions are used to change the flow of control of a program to a new instruction address specified as the target of branch and jump instruction. These instructions may be conditional or unconditional. For example,
JMP: is an unconditional branch to an instruction address. It may be used to implement simple loops.
JNE: (jump not equal) is a conditional branch instruction. This instruction checks the zero-flag register to determine, if two operands are equal or not (How the flags would be set? This will be explained in Block 4). The jump to the specified address of instruction will take place, only if the zero flag is not set.

Some of the conditional branch instructions are:
BRP X: branch to memory location X if the result of most recent operation is positive
BRN X: branch to memory location X if the result of most recent operation is negative

In the following example, conditional branch has been utilized in which, PC will be loaded with memory location 707 if the content of accumulator register (AC) is zero. And an unconditional JUMP instruction is to execute the program from a particular memory location 901.

```
DR ← 0          ; Assign 0 to the memory branch register DR
AC ← M[X]       ; Read a value from memory location X and store in AC
BRZ 707         ; Branch to location 707 if AC is zero (Conditional branch)
ADD AC, DR      ; Add the content of DR to AC and store result to AC
…
JUMP 901        ; jump to memory location 901 for further processing.
```

SKIP: The SKIP instruction skips the next instruction to be executed in sequence. Hence, it increments the value of PC by one instruction length. The SKIP can also be conditional. For example, the instruction ISZ skips the next instruction only if the result of the most recent operation is zero.

Subroutine call and return instruction : The subroutine call instcution holds the opcode and address of the start of subroutine. On execution of a subroutine call instruction, first the return address, which is the addresss of the next instruction to subroutine call stored in PC is moved to the memory, and then PC is loaded with the subroutine address from the subroutine call instruction. Once subroutine execution is completed, program has to retun back to the calling program to execute the next instruction after the subroutine call instruciton, which was stored as return address. Therefore, all the subroutines end with the return instruction. A subroutine call is implemented as below:

SP ← SP-1	stack pointer is decreased by one
M[SP]← PC	PC content is pushed onto the stack to store return address
PC ← effective address	From the subroutine call instruction the effective address is moved to the program counter (PC) register.

While the return instruction would be executed as:

PC ← M[SP]	The stored return address is assigned to PC
SP ← SP+1	stack pointer is increased by one

In subroutine call, the return address is saved at one location. Once call function execution is over, PC is loaded with the return address. A typical example of subroutine call and return in the context of 8086 microprocessor is explain in Block 4 of this course.

9.2.3 Stored Program Organization

A computer is organized to have processor registers and memory unit. Memory holds the program instructions as well as the operands/data on which operation is to be performed. For example, in a 16-bit memory of size 4096 words, as shown in Figure 9.3, an instruction of size one word includes an opcode and one operand address. Since the size of the memory is 2^{12} or 4096 words, the operand address would be 12-bits and the remaining 4-bits would represent an opcode, which defines the operation to be performed. Operand address defined in instruction is the memory location which holds 16-bit data.

Accumulator is the processor register. The operations are performed on the memory content and the accumulator data. A detailed and enhanced example of this structure is given in Block 4, which provides details on 8086 microprocessor.

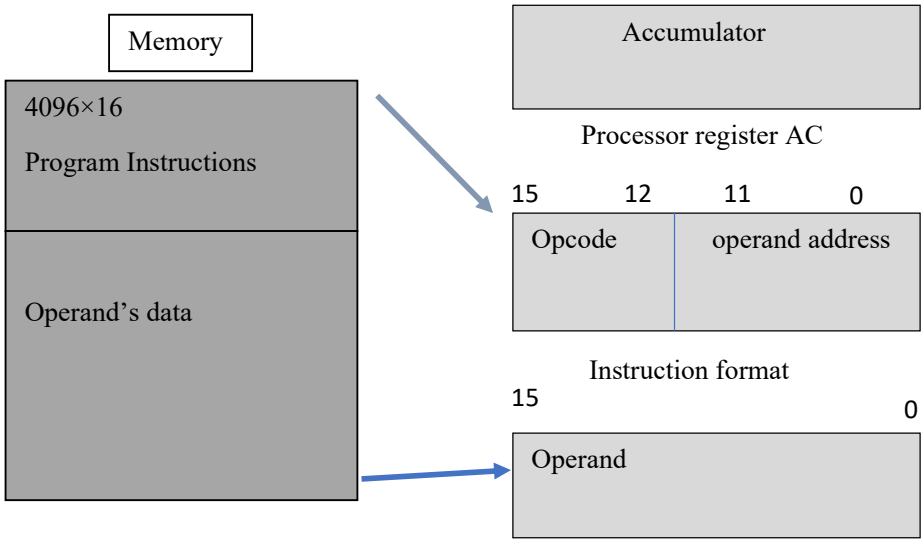


Figure 9.2: Schematic representation of stored program

☞ Check Your Progress – 1

1. What operation can be performed on A =0011 1001 to get a
   a. 1001 1100
   b. 0100 1110
   c. 0010 0111

.....

.....

.....

2. Define opcode and operand.

.....

.....

3. Consider that A contains 0101 1110 and B contains 0000 1111, then what would be the value of
   (a) Selective Set of A using B
   (b) Masking of A by B

.....

.....

9.3 NUMBER OF ADDRESSES AND INSTRUCTION SIZE

There are instructions set with zero address, one address, two address, three address, and RISC (Reduced Instruction Set Computers). What is the impact of number of

addresses in an instruction on instruction size and program? It will be discussed with the help of program that evaluates the arithmetic function:

$$A = (W+X) \times (Y+Z)$$

Zero address instruction: The instructions used for stack organized computers do not have the address field in the instruction. As you can see in the following instructions that no address is assigned in the instruction except for the PUSH and POP commands. The operands are implicit and are taken from the top of the stack. Hence, these are called Zero address instructions. The program to evaluate the value of A is given in Table 9.4.

Table 9.4: Zero Address instructions

PUSH W	The stack top position← W
PUSH X	The stack top position← X
ADD	The stack top position← W+X
PUSH Y	The stack top position← Y
PUSH Z	The stack top position← Z
ADD	The stack top position← Y+Z
MUL	The stack top position← (W+X)×(Y+Z)
POP A	A ← The stack top position

One address instruction: in these instructions, only one address field is present while the second operand is an implicit register operand - the AC accumulator register. The set of instructions that can solve the stated mathematical operation are discussed in

Table 9.5: One address instructions

Instruction	Function	Discussion
LOAD W	AC← M[W]	Data of Memory Address W is loaded in accumulator register
ADD X	AC←AC +M[X]	Data of Memory Address X is added with accumulator data
STORE A	M[A] ←AC	Accumulator data containing (W+X) is stored in Memory Address A
LOAD Y	AC← M[Y]	Data from Memory Address Y is loaded in accumulator
ADD Z	AC←AC +M[Z]	Data Memory Address Z is added with accumulator data
MUL A	AC←AC *M[A]	Memory address A is loaded in accumulator
STORE A	M[A]← AC	Accumulator data is stored in Memory address A

Table 9.5. You can observer that all the instructions in the Table 9.5 have just one operand address specified in the instruction. We can see, instructions are only with one address.

Two addresses instruction: Here, one instruction holds two addresses which may be memory address or processor register. As listed in the following table, all the instructions hold two addresses one of these is a processor register and the other is the memory location.

Table 9.6: Two-addresses instructions

Instruction	Function	Discussion
MOV R1, W	R1← M[W]	Data of Memory Address W is moved to register R1
ADD R1, X	R1←R1 +M[X]	Data of Memory Address X is added with data of register R1 and result is saved in R1
MOV R2, Y	R2← M[Y]	Data of Memory Address Y is moved to register R2
ADD R2, Z	R2←R2 +M[Z]	Data of Memory Address z is added with data of register R2 and result is saved in R2
MUL R1, R2	R1←R1 *R2	Data of registers R1 and R2 is multiplied and saved in R1
MOV A, R1	M[A]←R1	Results of R1 is moved to Memory address A

Three addresses Instructions: These instructions hold three addresses which may be processor register and/or memory. As shown in the following table, all the instructions hold three addresses.

Table 9.7: 3-address instructions

Instruction	Function	Discussion
ADD R1, W, X	R1← M[W]+M[X]	Data of memory location W and X are added and result is stored in processor register R1
ADD R2, Y, Z	R2←M[Y] +M[Z]	Data of memory location Y and Z are added, and result is stored in processor register R2
MUL A, R1, R2	M[A]←R1*R2	Data of processor registers R1 and R2 is multiplied and saved in memory location A .

Advantage of increasing the number of addresses: As you can observe from Tables 9.4, 9.5, 9.6 and 9.7 that number of instructions that are required to perform arithmetic operation decreases.
Disadvantage of increasing the number of addresses: The number of bits required to define an instruction keeps on increasing, thus, increasing the length of an instruction.

Reduced Instruction Set computers (RISC): RISC instructions use three processor registers for operations that perform computations, whereas one memory address and one processor register for the input/output instructions, such as load and store instructions. Instructions performing computational operation do not use memory. The following table illustrate a program based on RISC machine.

Table 9.8: reduced instruction set

Instruction	Function	Working
LOAD R1, W	R1← M[W]	Data of Memory Address W is loaded in processor register R1
LOAD R2, X	R2← M[X]	Data of Memory Address X is loaded in processor register R2
LOAD R3, Y	R3← M[Y]	Data of Memory Address Y is loaded in processor register R3
LOAD R4, Z	R4← M[Z]	Data of Memory Address Z is loaded in processor register R4
ADD R1, R1, R2	R1←R1+R2	Data of processor register R1 and R2 are added and result is stored in R1
ADD R3, R3, R2	R3←R3+R4	Data of processor register R3 and R4 are added and result is stored in R3
MUL R1, R1, R3	R1←R1* R3	Data of processor register R1 and R3 is multiplied added and result is stored in R1
STORE A, R1	M[A]←R1	Final result is stored from register R1 to memory address A

*LOAD: to load data from memory to processor register

*STORE: To store data from processor register to memeory

Advantages of RISC instruction set:

- Memory access is limited to load and store instruction only
- Even though three address instructions are used, however as most of the operands are registers, therefore instruction length is small.
- Single cycle executable instructions
- All operations are performed within processor registers

9.4 INSTRUCTION SET AND FORMAT DESIGN ISSUES

In Figure 9.1 of Section 9.2 an example instruction format is shown. This instruction format stores only one operand address, which is stored in the lowest 24 bits (bit 0 to bit 23) for the instruction. In case, this address is a direct operand address, then such an instruction format may support only 2^{24} = 16 M words memory. This computation assumes that each memory address is an address of one memory word. The opcode size is 7 bits (bit 24 to bit 30). Therefore, in general, there would be 2^7= 128 possible operation codes for this machine. The most significant bit (bit 31) is an addressing mode bit, which in this instruction format specifies the direct or indirect memory addressing mode. An instruction of a computer can have several addressing modes, which are explained in the next section. Thus, in an instruction format, there are three components: opcode, operand and the addressing mode. Hence, instruction length depends on the number of bits allocated to each component. The following are the issues relating to the instruction format design:

Instruction Length: Instruction length is critical in instruction format. There is trade of in smaller vs longer instructions.

a) More operands in one instruction will result into smaller programs, as discussed three address instructions have smaller program in comparison to instructions having lesser number of addresses. However, it may increase the length of an instruction.
b) Addressing modes: Its length is very crucial, as addressing modes give the flexibility of implementing different function.

Bits allocation to operand and opcode: Bits allocated to operand depends on the addressing mode used. For example, register addressing would require lesser number of bits than a memory address. Even the number of opcode depends on the flexibility, for example, an instruction set may have many different add operation codes for different addition operations. For example, a machine may have several addition operations like memory and one immediate operand, one memory and one register operand, two memory operand etc. each being assigned a different opcode. The addressing mode bits can facilitate bringing down such instructions. RISC computers use only register addressing (except for memory read and write instruction), thus, may simplify the instruction format.

Addressing mode: The bits allocated to addressing mode depends on the number of addressing modes used in the instruction set. The number of bits allocated will be high if number of addressing modes been used are high.

Variable length instruction: Computers use several different types of instruction formats. In certain machines these formats can be of different length. For example, an instruction format using one register operand may be a smaller format, whereas an instruction format using one direct memory operand would be of different instruction length. Complex Instruction sets comes under variable length instructions, abbreviated as CISC.

☞ Check Your Progress – 2

1. A computer with memory unit 512K words of 32 bits each. The computer has 32 registers. A binary instruction code is stored in one word memory, where instructions consist of an indirect bit, an opcode, a memory operand address and a register address.
   - Find the bits of opcode, register address, and memory address.
   - Draw the instruction word format.
   - Find out the bits required for the address and data input of the memory?

   ……
   ……

2. Give one instruction for each of the following:
   i. Zero address
   ii. One address
   iii. Two address
   iv. Three address

   ……
   ……

9.5 ADDRESSING SCHEMES

There is concept of addressing schemes which can be used by the programmer to get flexibility to do task. There are a number of addressing modes. A machine can support a number of these addressing modes.

Implied mode: In this mode, the operand is an implied operand for the given instruction. For example, CPLA is an instruction, which complements the accumulator register. The instruction does not contain the address of the operand register; however, it has one implied operand, i.e. accumulator register. Few other examples of implied addressing mode instructions are: INCA and DECA, where accumulator content is processed.

INCA Increments accumulator register content by one
DECA Decrements accumulator register content by one

9.5.1 Immediate Addressing

In this, the instruction holds the operand on which operation is to be operated, is called direct immediate operand. For example, the following instruction loads the value 20 in the accumulator register. The # sign in this instruction indicates that value 20 is an immediate operand. Thus, in immediate addressing mode an operand is part of the instruction. Suh addressing modes are very useful, when you want to initialize a register to a constant value.

Load AC, #20 Loads an immediate value *20* accumulator register.

Load R, #30 Loads an immediate value *30* in processor register R.

9.5.2 Direct Addressing

In this, the instruction specifies the memory address, where the operand is stored. This is one of the most fundamental addressing modes and is present in most of the machines, including RISC machine. For example, the following example, the content of memory location 200 would be loaded in the accumulator register.

Load AC, 200

In this instruction, memory address 200 holds the operand which is stored in the processor register accumulator. Therefore, instruction address is the effective address. Figure 9.4 illustrates this instruction at location 20. As per Figure 9.4, the location 200 contains the operand 350.

Thus, Effective Address of operand = 200
The value of operand which would be loaded in AC = 350.

Similarly, for an instruction *Load R, 350* in Figure 9.4, the effective address would be 350 and the operand value 10 that is stored in the memory address 300 would be loaded in processor register *R*.

9.5.3 Indirect Addressing

In this addressing mode, the address given in the instruction is the memory address where effective address of operands is stored. For example, consider following instruction in the Figure 9.4:

Load AC (200)

This instruction is stored at memory location 21, as shown in Figure 9.4. The instruction address 200 contains 350 which is the effective address of the operand. The operand stored at 350 is 10. Thus, in the indirect addressing mode for the given example,

Effective Address of operand = Content of location (200) = 350

The value of operand which would be loaded in AC = 10.

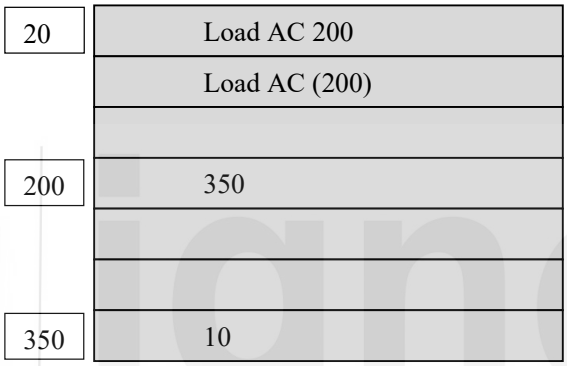


Figure 9.4: Schematic representation of Direct and Indirect addressing modes

9.5.4 Register Addressing

Operands are in register that reside within CPU. For example, consider the instruction:

LD R1

Which loads the content of register R1 to the accumulator register (AC). In this instruction, R1 is a processor register. In case, this instruction is executed on the machine shown in Figure 9.5, the effective address in this case is the address of register R1 itself. On execution of this instruction the content of R1, which is 201, will be loaded in the AC. Therefore, the content of the AC would be 201, after execution of this instruction.

9.5.5 Register Indirect Addressing

In the register indirect addressing, the instruction holds the processor register which carries the address of the operand in memory. For example, the instruction

LD (R1)

In this instruction the operands from memory location whose address in in R1 register is loaded in the accumulator.

AC ← M[R1]

For example, given the values stored in different registers and memory locations, as shown in Figure 9.5, the instruction LD(R1), which is a register indirect addressing mode will be interpreted as follows:

- The register R1 contains the value 201, therefore, the effective address of operand is memory address 201, i.e. EA=201
- The operand value stored in memory address 201 is 175. Hence, content of memory location 201, which is 175 is loaded in AC register.

Therefore, content of the AC after execution of this instruction would be 175.

9.5.6 Relative Addressing Scheme

In this mode, the operands are stored at the memory address obtained by adding address given in the instruction with the current program counter (PC) location. For example, the instruction:

LD $ADR ; ADR is the value of the operand address field of instruction.

This instruction results in the operation: AC ← M[PC+ADR], which means that the operands are located at memory address (effective address) PC+ADR. Assuming this is the instruction given in location 100 of the Figure 9.5, which can be written as:

LD $100 ; Please note ADR=100 in this instruction.

The effective address is calculated as: content of PC + ADR of the instruction. Since the current instruction is at PC value 100, on fetch of this instruction PC would be incremented to the address of next instruction, which is 102. Thus, the effective address would be 102+100 = 202, which means that operand is at the memory location 202.

EA =202

Therefore, on execution of this instruction, the content of memory location 202, which is 130, would be loaded in the accumulator.

Therefore, content of the AC after execution of this instruction would be 130.

9.5.7 Base Register Addressing

In the base register addressing scheme a base register and an offset from this register is specified in the instruction. Effective address is found by adding the base register content with the address part of the instruction. For example,

LD BR, ADR

The instruction, as given above, assumes that the instruction specifies the base register (BR in this instruction) and offset from the base register in the address part of the instruction (ADR). Assuming that the instruction at the location 100 is using base register addressing, the instruction can be written as:

LD BR, 100

Since, the value of the base register BR=100 and ADR=100. The effective address would be:

EA=BR+ADR = 100+100 = 200

Therefore, on execution of the instruction the content of the memory location 200, which is 170, would be loaded in the accumulator.
AC is loaded with 170

Therefore, content of the AC after execution of this instruction would be 170.

9.5.8 Indexed Addressing Scheme

In the indexed addressing scheme an address of a memory location is defined and the value of an index register, which represents an offset, is added to it to find the effective address of an operand. This addressing scheme is very useful for accessing an array, where an index register points to offset in an array of values. For example, the following instruction uses the index register XR with the original offset of data is stored in ADR, which is the operand field of the instruction.

LD ADR(XR)

The following operation defines the operation performed by the instruction.

AC ← M[ADR+XR]

For example, as ADR portion of the instruction is 100 and if the index register is referring to 200th element of an array that is XR=200, then effective address would be:

EA =100+200 = 300

Therefore, on execution of this instruction, the content of Memory location 300 would be loaded in the accumulator. Thus, AC will be loaded with 140 (Refer to Figure 9.5).

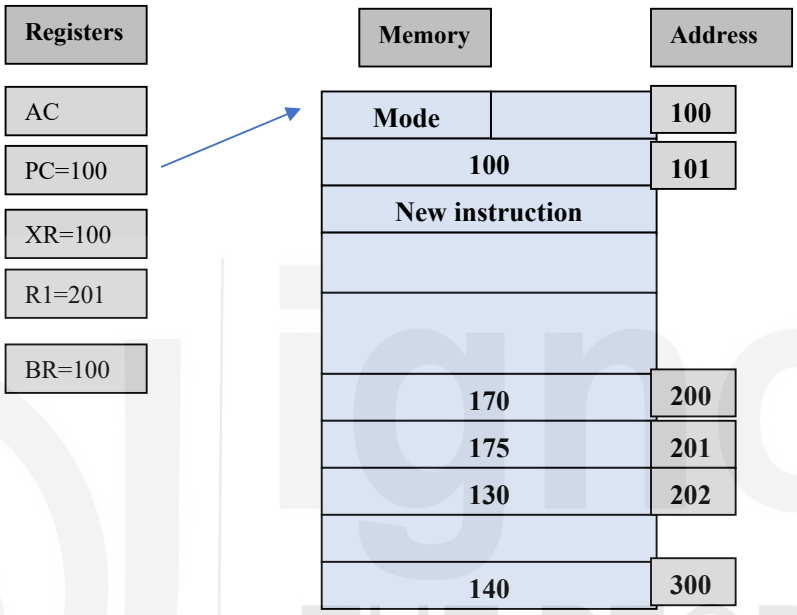


Figure 9.5: An example for addressing modes

9.5.9 Stack Addressing

Stack organization operated on Last in First out (LIFO), e.g. as you arrange books one above other in a bookshelf, while taking out the books last placed book will be lifted first. A stack is collection of finite number of words, as shown in Figure 9.6. It is a 64-word stack of memory locations. A register called Stack Pointer (SP) is used to hold the word address of top of the stack. Data register DR holds the data to be written or read from the stack. Where FULL and EMPTY are one –bit registers to find out if the stack is full or empty. Where data bit FULL is 1 when stack is full, and EMPTY bit is 1 when stack is empty. A new data is inserted in the stack using PUSH, when stack is not full, i.e. FULL=0, the PUSH operation is shown below:

SP ← SP+1	Stack pointer is incremented to an empty location
M[SP] ←DR	Data is written into stack top

Whereas, POP deletes a data item from the top of the stack and puts it in DR register:

DR ←M[SP]	data of top-most stack location is read to DR
SP←SP-1	Stack pointer is decremented by 1

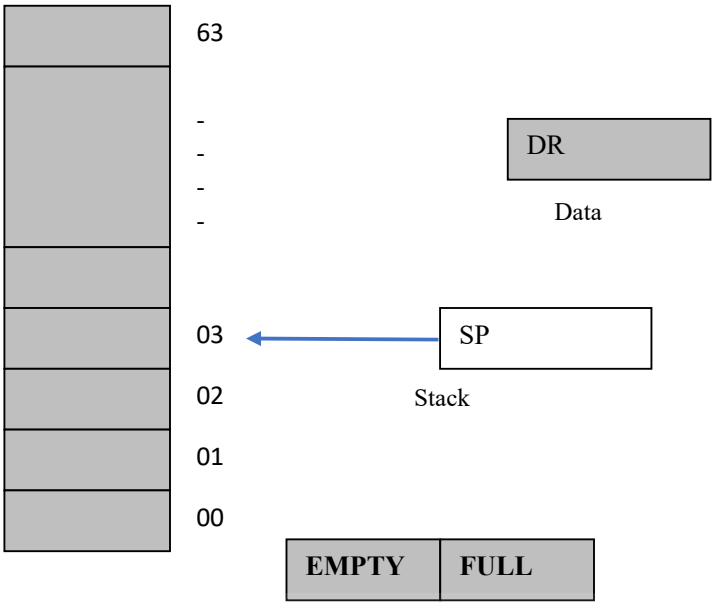


Figure 9.6: Stack organization

Check Your Progress 3:

1. Find the effective address in case of each addressing modes using the Figure 9.7.

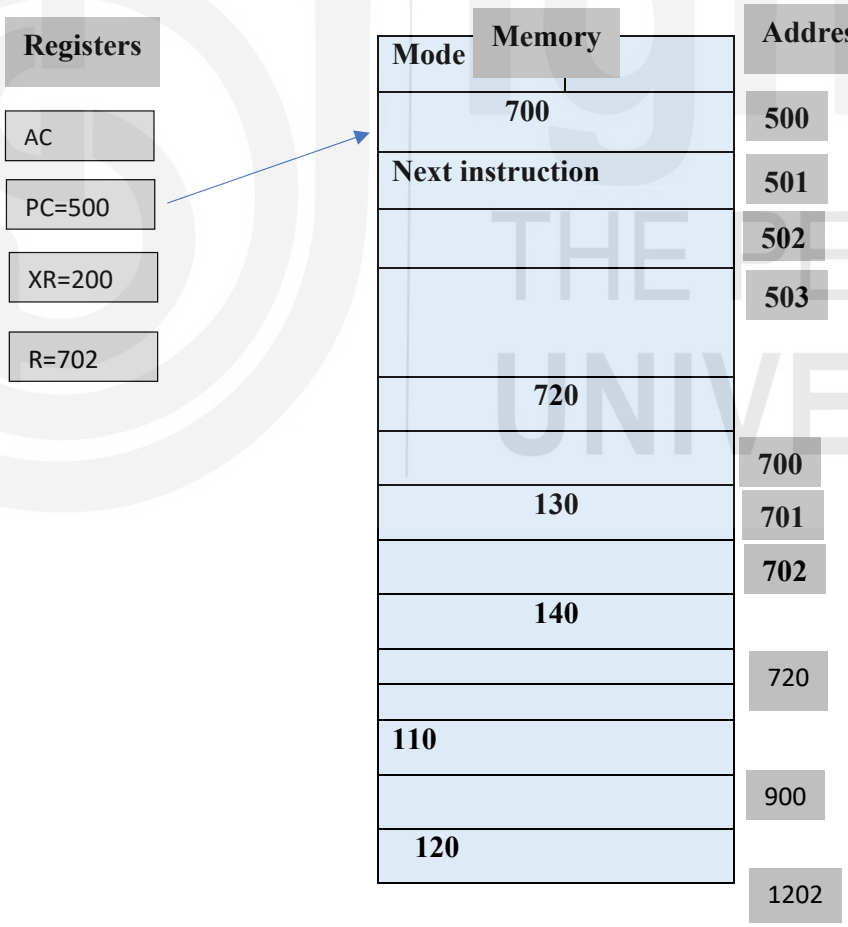


Figure 9.7: The Example Data

You may assume that the instruction is at location 500 and 501. Please note AC is accumulator register; PC is Program counter register; XR is index

register; and R is any processor register. The address of the instruction to be executed, is stored in processor register program counter.

.....

.....

.....

Q 2: which instruction set is with zero address?

.....

.....

.....

.....

9.6 SUMMARY

In this unit, basics of instructions such as operand data types, instruction length, and instruction set has been discussed. The instruction format with details of operands and opcode had been explained. In addition to that, various addressing modes, and related instruction size (w.r.t. number of addresses) has been discussed. Depending on the addressing modes, the number of addresses in one instruction and the program complexity has been explained. The flexibility provided to the programmer because of the addressing modes has also been deliberated.

9.7 ANSWER TO CHECK YOUR PROGRESS

Check Your Progress 1

1. a) cir r 1-time
   b) cir r 2-time
   c) cir r 3 –times
2.
   - *Opcode*: opcode is to decides the operation to be performed on the data.
   - *Operand*: operands are the data on which operation is to be performed

3. Consider that A contains 0101 1110 and B contains 0000 1111
   Selective set : A OR B 0101 1111 (to set LSB 4)
   Masking A AND B 0000 1110 (masked MSB 4 bits)

Check Your Progress 2

1 Memory unit =512K

$= 2^{19}$

Hence, size of memory address =19 bits
Register address for selecting one of 32 registers $=2^5$
= 5 bits

Indirect bit =1

Opcode is : 32-(19+5+1)=7 bits

Indirect bit	Opcode	Register	Address
31	30 24	23 19	18 0

1. The instructions are as follows:

i. Zero address: In stack organised computers after two PUSH instructions, which will put two operands on a stack, the zero address ADD instruction will add the content of the top two operands.

ii. One address: In an accumulator-based machine, which uses one of the operands as AC and stores result in AC. The one address instruction: ADD X will add the content of memory location X to the AC register and result of addition will be in AC.

iii. Two address: An instruction like ADD R1, R2 would add registers R1 and R2 and store the result in R1 register.

iv. Three address: An instruction like ADD R3, R1, R2 would add registers R1 and R2 and put the result in R3 register.

Check Your Progress 3

Q1:

Addressing mode	**Effective address**	**Accumulator content**	**Remarks**
Immediate	500	700	Instructions holds the operand data
Direct	700	720	Instruction holds the memory address where operand is stored
Indirect	720	140	Instruction holds the memory address where the operand address is stored
Relative Address	1202	120	Relative address holds the operands at: PC+ instruction content (502+700)
Index Address	900	110	Index address holds the operands at : XR+700 (Index register is added with instruction content)
Register	-	702	Operand is stored in CPU Register

Register indirect	702	130	Register holds the Memory address where operand is stored

Question 2: Stack organization instructions PUSH and POP

ignou
THE PEOPLE'S
UNIVERSITY

UNIT 10 REGISTERS, MICRO-OPERATIONS AND INSTRUCTION EXECUTION

Structure **Page No.**

10.0 Introduction
10.1 Objectives
10.2 Register Organization
10.2.1 Programmer Visible Registers
10.2.2 Status and Control Registers
10.3 Micro-operation concepts
10.3.1 Register Transfer Language, Bus and Memory Transfers
10.3.2 Register Transfer Micro-Operation
10.3.3 Arithmetic Micro-operations
10.3.4 Logic Micro-operations
10.3.5 Shift Micro-operations
10.4 Instruction Execution and Micro-operations
10.4.1 Relationship of Timing and Control, Memory reference instructions and Input-output Interrupts to instruction execution
10.4.2 Instruction Cycle
10.4.3 Interrupt Cycle
10.5 Instruction Pipelining
10.6 ALU Organization
10.6.1 A Simple ALU Organisation
10.6.2 A Simple ALU Design
10.7 Arithmetic Processor
10.8 Summary
10.9 Solutions/ Answers

10.0 INTRODUCTION

In the previous unit, you have gone through the concepts relating to various types of instructions and operands that a computer can have. The main task performed by the CPU is the execution of the instructions. This Unit focusses on the process of execution of these instructions by the CPU. This Units tries to answers the following two questions regarding instruction execution.

What are the steps required for the execution of an instruction? And

How are these steps performed by the CPU?

Execution of an instruction can be divided into sequence of steps, together they constitute an instruction execution sequence, called instruction cycle. Each of these steps can be termed as a micro-operation. A micro-operation is the smallest operation performed by the CPU. These operations put together execute an instruction.

For answering the second question, youmayrecall the basic structure of a computer. The CPU of a computer consists of an Arithmetic Logic Unit (ALU), the Control Unit (CU) and operational registers. We will be discussing the register organisation and ALU in this unit, whereas the control unit organisationis discussed in next unit.

In this unit we will first discuss the basic CPU structure and the register organisation in general. This is followed by a discussion on micro-operations that include register–transfer, arithmetic, logic and shift micro-operation and their implementation, which

forms the basis of design of a ALU. The discussion on micro-operations will gradually lead us towards the discussion of an ALU structure. The unit will also discuss about the arithmetic processor, which are commonlyused for floating point computations.

10.1 OBJECTIVES

After going through this unit, you should be able to:

- describe the register organisation of the CPU;
- define what is a micro-operation;
- discuss an instruction execution using the micro-operations; and
- describe the basic organisation of ALU;
- discuss the requirements of a floating point ALU;
- create simple arithmetic logic circuits.

10.2 REGISTER ORGANISATION

The number and the nature of registers is a key factor that differentiates among computers. For example, Intel Pentium has about 32 registers. Some of these registers are special registers and others are general-purpose registers. Some of the basic registers in a machine are:

- All von-Neumann machines have a program counter (PC) (or instruction counter IC), which is a register that contains the address of the instruction that is expected to be executed next.
- Most computers use special registers to hold the instruction(s) currently being executed. They are called instruction register (IR).
- There are a number of general-purpose registers, which can be used for arithmetic computations or any other purpose.
- Memory-address register (MAR) holds the address of next memory operation (load or store).
- Memory-buffer register (MBR) or Memory data Register (MDR) holds the content of memory operation (load or store).
- Processor status bits indicate the current status of the processor. Sometimes it is combined with the other processor status bits and is called the program status word (PSW). Some processors also use flags register, which store different flags set by the processor like carry flag, overflow flag, zero flag etc.

The CPU registers have the following characteristics:

- CPU can access registers faster than it can access main memory.
- Register addressing requires less bits in the instructions for addressing than that of memory addressing. For example, for addressing 256 registers you just need 8 bits, whereas the memory size of 1MB would requires 20 address bits, a difference of 60%.
- Compilers tend to use a small number of registers, as large numbers of registers are difficult to use effectively. A general good number of registers is 32 in a general machine.
- Registers are more expensive than memory but far less in number.

From a user's point of view, computers have two different kinds of registers. These are:

Programmer Visible Registers: These registers can be used by machine or assembly language programmers while programming. A good program minimizes the references to main memory.

Status Control and Registers: These registers cannot be used by the programmers but are used to control the CPU or the execution of a program.

Different chip designer companiesuse some of these registers interchangeably; therefore, you should not stick to these definitions rigidly. Yet this categorization will help in better understanding of register sets of a machine. Therefore, let us discuss more about these categories.

10.2.1 Programmer Visible Registers

These registers can be accessed using machine language. In general,there are four types of programmer visible registers.

- General Purpose Registers
- Data Registers
- Address Registers
- Condition Codes Registers.

A comprehensive example of registers of 8086 is given in Unit 1 Block 4.

The general-purpose registers as the name suggests can be used for various functions. For example, they may contain operands or can be used for calculation of address of operand etc. However, to simplify the task of programmers and computers dedicated registers can be used. For example, registers may be dedicated to floating point operations. Such dedication may lead to design of data and address registers.

The data registers are used only for storing intermediate results or data and not for operand address calculation.

The address registers are used for address computation. Some dedicated address registers are:

Segment Pointer : Used to point out a segment of memory.
Index Register : These are used for index addressing scheme.
Stack Pointer : Points to top of the stack when programmer visible stack addressing is used.

One of the basic issues with register design is the number of general-purpose registers or data and address registers to be provided in a microprocessor. The number of registers also affects the instruction design as the number of registers determines the number of bits needed in an instruction to specify a register reference. In general, the optimum number of registers in a CPU is in the range 16 to 32. In case registers fall below the range then more memory reference per instruction on an average will be needed, as some of the intermediate results then must be stored in the memory. On the other hand, if the number of registers goes above 32, then there is no appreciable reduction in memory references. However, in some computers hundreds of registers are used. These systems have special characteristics. These are called Reduced Instruction Set Computers (RISC) and they exhibit this property. RISC computers are discussed in a later unit.

What is the importance of having less memory references? As the time required for memory reference is more than that of a register reference, therefore the increased number of memory references results in slower execution of a program.

Register Length: An important characteristic related to registers is the length of a register. Normally, the length of a register is dependent on its use. For example, a register, which is used to calculate address, must be long enough to hold the maximum possible addresses. If the size of memory is 1 MB than a minimum of 20 bits are required to store an instruction address. Please note how this requirement can be optimized in 8086 in the block 4. Similarly, the length of data register should be long enough to hold the data type it is supposed to hold. If the length of a data register is half of the size of data, then it is possible that two consecutive registers, rather than on single register, are used to store the data.

10.2.2 Status and Control Registers

For control of various operations several registers are used. These registers cannot be used in data manipulation; however, the content of some of these registers can be used by the programmer.Almost all the CPUs have a status register, a part of which may be programmer visible. A register which may be formed by condition codes is called condition code register. Some of the commonly used flags or condition codes in such a register may be:

Flag	Comments
Sign flag	This indicates whether the sign of previous arithmetic operation was positive (0) or negative (1).
Zero flag	This flag bit will be set (contain a value 1) if the result of the last arithmetic operation was zero.
Carry flag	This flag is set, if a carry results from the addition of the highest order bits or borrow is taken on subtraction of highest order bit.
Equal flag	This bit flag will be set if a logic comparison operation finds out that both of its operands are equal.
Overflow flag	This flag is used to indicate the condition of arithmetic overflow.
Interrupt	This flag is used for enabling or disabling interrupts. Enable/ disable flag.
Supervisor flag	This flag is used in certain computers to determine whether the CPU is executing in supervisor or user mode. In case the CPU is in supervisor mode it will be allowed to execute certain privileged instructions.

Figure 10.1: Flags or conditional codes

These flags are set by the CPU hardware while performing an operation. For example, an addition operation may set the carry flag and zero flag; or on a division by 0 the overflow flag can be set etc. These flags or conditional codes are tested by a program while performing typical operations like conditional branch operation. The condition codes are collected in one or more registers. RISC machines have several sets of conditional code bits. In these machines an instruction specifies the set of condition codes which is to be used. Independent sets of condition code enable the provisions of having parallelism within the instruction execution unit.

The flag register is often known as Program Status Word (PSW). It contains condition code plus other status information. There can be several other status and control registers such as interrupt vector register in the machines using vectored interrupt etc.

☞Check Your Progress 1

1. What is an address register?

   ……

   ……

   ……

2. A machine has 20 general-purpose registers. How many bits will be needed for register address of this machine?

   .....

   .....

   .....

3. What is the advantage of having independent set of conditional codes?

   .....

   .....

   .....

4. Can you store status and control information in the memory?

   .....

   .....

   .....

10.3 MICRO-OPERATION CONCEPTS

We have discussed the general introduction to register set. A computer executes an instruction using the arithmetic logic unit and control unit in several steps. Each of these micro steps of instruction execution is elementary in nature and is termed as a micro-operation. In general, a micro-operation should be executed in a single clock pulse by the ALU or bus transfer unit using the data stored mostly in registers.

A machine instruction is equivalent to an assembly language instruction, with the main difference being that assembly instructions use mnemonics, while machine instructions use opcode, operand addresses etc. Thus, for simplicity of explanation, we will use assembly language instructions rather than machine instructions. It may be noted that a high-level language program, first is converted to machine instructions and is executed thereafter. For example, a C programming language expression A=A+B; may require the following sequence of machine/assembly instructions on a hypothetical machine that uses AC and DR registers:

```
LDA AC, A  ; Load the content of memory location A to AC register.
LDA DR, B  ; Load the memory operand B to DR register
ADD AC, DR ; The content of AC and DR is added and stored in AC
STR  A, AC ; Store the content of AC to memory location A.
```

Each of the above instructions is required to be executed by the computer. Consider the instruction LDA AC, A how will it be executed? A von Neumann machine may execute instructions in the following steps:

Step 1: Get the instruction from memory to the Instruction Register (IR): This step itself will consists of several micro-steps, which will require transfer of content among registers and using the bus.

Step 2: Decode the instruction: This will be the job of control unit (CU) and it will issue the necessary set of control signals. This step will be discussed in Unit 11.
Step 3: Fetch the operands, if needed, in the processing unit registers.
Step 4: Execute the instructions using arithmetic logic unit (ALU) and store the results back to processing unit registers.
Step 5: Store the result back to memory, if needed.

Please note that each of these steps may require several micro-steps and those micro-steps are the micro-operations. May of these steps are due to the specific functions of registers, for example, the Program counter (PC) register stores the address of instruction that is to be executed next. Thus, in order to get the next instruction from the memory, you are required to transfer this information to the register that is used as an memory address register. Similarly, instruction once fetched from memory may be in a data register, since IR is used as input for decode operation, the instruction must be sent to IR register. The micro-operations that are used for representing data transfer between two registers or one register and one memory location using the bus are termed as register transfer micro-operations.

In addition, to register transfer, instruction execution also involves the use of arithmetic logic unit. Some of these operations which are performed by ALU on register data are add, subtract, increment, decrement etc. These are called arithmetic micro-operations. In addition, ALU can also be used to perform the logic operations, like AND, OR, NOT etc., on the data of registers. These are called the logic micro-operations. Further, shifting of data to left or right by one bit are used for many useful operations like multiplication or division. Such micro-operations may be termed as shift micro-operations.

These microoperation can be written using a register/memory transfer language which is described next.

10.3.1 Register Transfer Language, Bus and Memory Transfers

Register transfer language can be used to represent various micro-operations. In addition, this language also can be used to represent bus and memory transfer. In this language the symbol ← is used to indicate transfer of content. For example, if two registers are named R1 and R2, then a register transfer micro-operation R1←R2 implies that all the bits of register R2 are to be transferred to register R1. This operation will be feasible only if both the registers are connected through a data path and consists of same number of bits. Some of the basic rules/convention of register transfer language are listed below:

1. Naming of Register: Register would be named using capital alphabets and digits. The first character of the name should be alphabet. Figure 10.2 shows the bit organization and naming of register parts with the help of an example register IR, which is having 16 bits. A register can be partitioned in bytes and each byte may be assigned a name. For example, Figure 10.2 represent IR register of 16 bits, which can have two 8-bit partitions. You can refer to each of these bytes separately if so desired. For example, the lower byte of IR can be referred to as IR(L) and higher byte of IR can be referred to as IR(H) (Please refer to Figure 10.2)

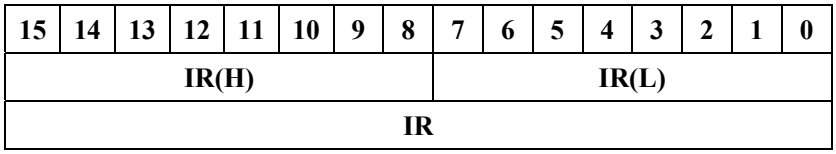


Figure 10.2: Register Naming and bit ordering

2. Information transfer from one register to another is designated by the symbol ←, which replaces the content of the destination register by the content of the source register, as explained earlier. The content of source register remains unchanged.

3. The control signal which enables the process of register transfer is shown with the help of a Boolean control function. This feature is very useful as operation can be controlled. For example, if *c* is a Boolean control variable that is it can have a value of 0 or 1,which controls the transfer of content from R2 to R1, then it will be represented as:

c: R1← R2 ; Please note this transfer will occur only if *c* = 1.

4. The micro-operations written on the same line are executedin the same clock time. However, such micro-operations should be free from conflict. The following example depicts a conflict in micro-operations; thus, these micro-operations should not be executed in parallel. What is a conflict in micro-operations? It is explained with the help of following example.
Example: Consider that a register R1 is to be incremented and it is also to be loaded with the content of IR register, then if you represent the following parallel micro-operations would be in conflict:
c: Rn← Rn+1, Rn←IR
These two micro-operations would update the register Rn at the same time, so one of these updated values would be lost.

5. All the register transfers occur during the falling edge transition of the clock. For more details, you may refer to further readings.

Use of Bus in register transfer

As explained earlier, a computer system may have about 16-32 or even more (in case of RISC) registers. A computer system, in general, uses an internal Bus, which consists of bus lines or wires, to transfer register data bits between two registers at a time. The control unit selects these two registers – one as the sources register and second as the destination register. Thus, the register transfer operation can be expanded as follows:

c: R1 ← BUS ← R2

The control unit causes the register R2 to put its content (one bit on one bus line). In addition, it also enables the load operation on the R1 register. Thus, the data bits of R2 are transferred to data bits of R1.

Memory Transfer

A von Neumann machine stores the program and data in the main memory of a computer. Therefore, instruction execution requires reading and writing operations from the memory. The main memory and the processing units of a computer are connected through the system bus, which includes address, data and control bus. The address bus is used to select the specific RAM word from the memory, which in turn is transferred over the data bus. The control unit controls the entire process of data transfer by sending control signals through control bus. The two basic operations performed on the memory are Read and Write operations.
Memory Read: Read operation on the memory requires the information about the location of the memory, which is to be read. The processor decides which data word or instruction word from the memory is to be read. The address of that memory word is put in an address register (AR) and then applied on the address bus, simultaneously enabling the memory read operation using the control bus. This causes the selected word from the memory unit to be placed on the data bus part of system bus. Control unit also enables data register (DR) to accept the input from the data bus. Thus,

completing the memory read operation. The following micro-operation shows this memory read operation (please note the use of [ ] symbol to represent memory):

mr: DR ← [AR] ; Content of memory addressed by AR is send to DR register

Memory Write: Write operation on the memory requires– the location of the memory, which is to be written and the content, which is to be written. The processor puts the address of memory word, which is to be written, to an address register (AR) and then applies this address on the address bus, simultaneously loading the content of data to be written, which may be stored in a data register (DR) and enabling the memory read operation using the control bus. This causes the selected word on the memory unit to accept the data bus. control unit also enables data register (DR) to accept the data from the data bus. Thus, completing the memory write operation. This operation can be represented using following micro-operation:
mw: [AR] ←DR ;Write the content of memory addressed by AR register by DR

Please note, in this micro-operation the memory location, as addressed by AR, is written into. The content of AR and DR registers remains unchanged

10.3.2 Register Transfer Micro-operations

Register transfer micro-operation is one of the basic micro-operations. Two basic requirements for such a transfer are:

- There should exist a direct path, such as internal bus, from sender register to receiver register. It may be noted that number of bus lines and the sizes of sender and receiver register should be the same.
- Since a micro-operation is proposed to be completed in a single clock pulse, therefore, all the data bits on the receiver register should be loaded at the same time. Thus, the receiver register must support parallel loading of bits (Refer to Unit 4 Block1).

Following are some of the examples of register transfer micro-operations:

```
R1← R2      ; Transfer the content of R2 register to R1 register
AR← PC      ; Transfer the content of PC register to AR register
```

10.3.3 Arithmetic Micro-operations

Arithmetic micro-operations are performed by the arithmetic logic unit on the data stored in the processor registers. The output is also saved in a processor register. Following are some common arithmetic micro-operations:

```
AC←AC+DR    ; Addition of AC and DR, result in AC
AC ← AC-DR  ; Subtraction of DR from AC, result in AC
AC ← AC+1   ; Increment AC, result in AC
AC ← AC-1   ; Decrement AC, result in AC
```

An adder subtractor circuit as shown in Unit 3 of Block 1 may be used to perform subtraction in machines through complement and add operations. It is represented as:

$$R3 \leftarrow R1 \; 0 \; R2$$

Please note that this micro-operation can also be represented as:

$$R3 \leftarrow R1 + R2' + 1$$

Why? R2′ is complement of R2, on adding 1 to it you get 2's complement of R2. Thus, both the above micro-operations are equivalent.

An addition and subtraction micro-operations can be implemented using an ALU that supports simple arithmetic operations, as discussed in section 10.4. Assuming that this ALU does implement the addition, subtraction, simple logic and shift micro-operation of fixed-point numbers, how will this machine implement multiplication and division operations on fixed point and floating-point numbers? In such a machine these operations can be implemented with the help of programs, which may use micro-operations like addition, shift and so. This kind of implementationrequires that operations like fixed point multiplication and division be implemented using several micro-operation steps using several micro-instructions. Thus, fixed point multiplication and division and floating point arithmetic instructions cannot be considered as micro operations for this kind of machine.

10.3.4Logic Micro-operations

A computer system is a binary device. Itperforms binary operations using bitwise Boolean operations. These bitwise Boolean operations that are implemented in the ALU forms the logic micro-operations.Three most common logic micro-operations are AND(.), OR(+) and NOT(~). Other logic micro-operations, which may be implemented in different computers can be XOR, NAND and NOR. For example, you can perform bit wise AND of two registers R1 and R2 using the AND micro-operation, which can be represented as:

R1 ← R1.R2

The result of the micro-operation, as given above, will be stored in the R1 register. A typical use of this micro-operation is shown in the following example.

Example 1: Assume that two four-bit registers R1 and R2 contains the data 1100 and 1010 respectively. What would be the output if following micro-operations are performed on these two registers:

i. R1 ← R1 . R2
ii. R1 ← R1 + R2
iii. R1 ←~R1

Solution:

i. 1100 .1010 = 1000
ii. 1100 +1010 = 1110
iii. ~1100 = 0011

Example 2: Consider a register A containing an 8-bit value 01010011. Find the value of register B and micro-operation, which can be used to set the upper four bits of the register A, while the lower four bit remains unchanged.
Solution: To set the upper four bits irrespective of the values of A while keeping the lower four bits unchanged, the register B can consist of value 11110000 and the micro-operation OR can be used, as shown below:

Register A	0	1	0	1	0	0	1	1
Register B	1	1	1	1	0	0	0	0
A OR B	1	1	1	1	0	0	1	1

Thus, in the output the upper four bits contains value 1, while lower four bits are same as that of register A.

Example 3: Use the same value of register A, as used in example 2. Find the value of register B and micro-operation, which can be used to clear the upper four bits of the register A, while the lower four bit remains unchanged.
Solution: To clear the upper four bits irrespective of the valueof A while keeping the lower four bits unchanged, the register B can consist of value 00001111 and the micro-operation AND can be used, as shown below:

Register A	0	1	0	1	0	0	1	1
Register B	0	0	0	0	1	1	1	1
A OR B	0	0	0	0	0	0	1	1

Thus, in the output the upper four bits contains value 0, while lower four bits are same as that of register A. This operation is sometimes also referred to as mask operation, where the upper four bits of the register A are masked out.

Example 4: Use the same value of register A, as used in example 2. Find the value of register B and micro-operation, which can be used to clear all the bits of a register.
Solution: To clear the entire register A, you can use register B same as that of register A and use XOR micro-operation, as shown below:

Register A	0	1	0	1	0	0	1	1
Register B	0	1	0	1	0	0	1	1
A XOR B	0	0	0	0	0	0	0	0

Example 5: Use the same value of register A, as used in example 2. Find the value of register B and micro-operation, which can be used to clear all the bits of a register.
Solution: One of the simplest ways to complement a register is to perform NOT micro-operation on register itself. An alternative native method would be to perform XOR with register B containing all 1's, as shown below:

Register A	0	1	0	1	0	0	1	1
Register B	1	1	1	1	1	1	1	1
A XOR B	1	0	1	0	1	1	0	0

10.3.5Shift Micro-operations

Shifting of bits of a register can be used for several useful functions in a computer, such as serial transfer of data, multiplication operation, division operation. A register consists of a linear sequence of bits, which can either be shifted towards the left direction or right direction. Shifting by one bit, irrespective of left or right shift, will result in one bit to move out of the shift register from one end and one bitwill be input to the register over the other end. Shift operations have been discussed in the Unit 9. Shift operations are of three basic types:

1. Logical Shift: In the logical shift, the input bit is kept as 0 and output bit is discarded.
2. Arithmetic shift: In arithmetic shift the sign of the number is kept the same
3. In circular shift the output bit is circulated to the input.

☞ Check Your Progress 2

1. Explain how the memory read and memory write micro-operations can be performed in a von Neumann machine

……

……

……

2. A simple ALU just performs the addition operation, logic operations and shift operations. Will multiplication on this machine be implemented as a micro-operation? Justify your answer.

……

……

……

3. Consider a register R1 contains 00010011. Select a suitable register R2 and sequence of logic micro-operations that can perform the following tasks:

(i) Reset the complete register R1, you must use this using Mask micro-operation and not using XOR.
(ii) Insert the 11001000 data into the register R1. You may use more than one micro-operation to do this task.

……

……

……

10.4 INSTRUCTION EXECUTIONS AND MICRO - OPERATIONS

To execute an instruction, a sequence of micro-operations needs to be executed. This sequence of micro-operations is essentially controlled by using a timing signal. In addition, an instruction execution involves operands. These operands can be stored either in the registers orthe memory of a computer. In other words, every computer has register reference and/or memory reference instructions. Finally, in general, input-output on a computer, as discussed in Block 2, may use input-output interrupts. The next sub-section discusses the importance of these concepts in the context of instruction execution. This is followed by discussion on instruction cycle and interrupt cycle.

10.4.1 Relationship of Timing and Control, Memory reference instructions and Input-output Interrupts to instruction execution

Timing and Control: The basic task of a computer system is to execute instructions. Each instruction is executed as a sequence of steps. The execution of instructions follows a typical sequence, which is termed as instruction cycle, which is discussed next sub-section. Thus, each instruction cycle consists of several steps, which are executed in a sequential orderto execute the instruction correctly. How will computer ensure that the exact sequence of instruction is followed?

As discussed in Unit 4, a computer system uses a clock for synchronization. The role of a clock is to continuously emit a sequence of clock pulses. In addition, Unit 4 also highlighted few important sequential circuits like registers, counters, multiplexer etc. Most of the sequential circuits change their state on the falling edge of the clock pulse. Thus, a particular action like loading of a register will be complete at the falling edge of the clock pulse. The next sequential action will be completed in the subsequent clock pulse and so on. For example, loading of a register may require one clock pulse, which may be followed by some other operation on the register in the next clock pulse. The counter is the circuit, which can be used the count the sequence of clock pulses. Thus, timing sequences are utilized to control the sequence of operations in a computer system. You may refer to the further readings for more details on these topics.

Memory Reference Instructions: The memory reference instructions access its operands from the memory. In general, these operands may use direct or indirect addressing schemes. In the direct memory access scheme, the address part of the instruction contains the direct reference of a memory operand, or in other words the

address part of the instruction contains the address of the operand. Thus, in order to process such instructions these operands are to be fetched from the memory to some data register of the processor. In the indirect memory reference, the instruction address is the operand address. Thus, two memory accesses would be needed to fetch such operands. First to get the address of the operand and second to get the operand from the memory.

It may be noted that a memory reference, in general, is slower than the reference to a register, as main memory is somewhat slower than the registers. However, with the use of fast cache memories, the difference in speed of the memory access vis-à-vis register access is being reduced. Many memory organizations have also been designed to reduce this speed gap. You may refer to the further readings for more details on these topics.

Input-output Interrupts: Interrupt is a condition, which may result in break in the sequence of instruction execution. Interrupts are used in computers for input-output as well as recovering from a condition that may cause an error, for example, division by zero. An Input-Output interrupt can be used for Input or output of data. For example, a program, while it is being executed by the processor, may have a command for reading of data from a keyboard. The processor would display the prompt for input of data for this program and may go on to execute other programs, while suspending this program as it needs the input. However, as soon as you enter the input from the keyboard, it may issue an interrupt to the processor, which in turn will process the input. More details on Interrupts are already given in Block 2. You may also refer to further readings for more details on interrupts.

Next section discusses the instruction cycle.

10.4.2 Instruction Cycle

As discussed in Unit 1, a von-Neumann machine having a basic set of registers executes instruction. These instructions consist of the following steps of execution:

Step 1: Get the Instruction from the memory to IR, the instruction register, let us call this operation as Fetch the Instruction (FI).

Step 2: Decode the Instruction, which is in the IR register, let us call this operation as DI.

Step 3: In this step operand address is converted to a direct address. are fetched, let us call this step as fetch the direct operand address (FoA).

Step 4: Perform the execution of instruction, let us call this step as execute instruction (ExI)

Step 5: Once execution of instruction is complete, check and acknowledge interrupt if an interrupt has occurred, let us call this step as check and acknowledge interrupt (CaI)

How are these steps can be translated to micro-operations? This section uses the register set as given in Section 10.2. In addition, it assumes the instruction format as given in Unit 9, where an instruction has three fields, viz. Indirect bit, opcode of 7 bits and one operand address of 24 bits. It may please be noted that the assumed instruction set has only direct memory and indirect memory addressing schemes. Further, only for the purpose of simplifying the discussion, it is assumed that memory reference and register reference is performed in almost equal time.

Step 1 (FI cycle): An instruction is available in the memory and program counter register (PC) points to this instruction, which is to be executed. Thus, in order to get the instruction from the memory, bus would be used along with memory address register (MAR). Thus, a sequence of operations would be needed to perform this operation. This sequence is represented with the help of a timing sequence using the timing control T1, T2, etc. Please note the micro-operation with timing control T1 will

be performed prior to micro-operations with timing control T2 and so on. Figure 10.4 lists the micro-operation sequence of FI cycle.

.. Transfer the content of PC to MAR.	T1: MAR ← PC
... Apply MAR on address BUS; Control unit enables the memory Read operation and DR is enabled to receive content on the data BUS. Thus, the content of memory location pointed by MAR is read to DR.	T2: DR ← (MAR)
... Perform the following two operations in parallel at time T3: o Increment PC so that it points to the next instruction to be executed. (PC is incremented by one memory word length, as it is assumed that each instruction is just one word long and memory address is a word address). o The instructionin DR is sent to IR to complete the FI cycle.	T3: PC←PC +1 : IR ← DR

Figure 10.4: Fetch cycle

Step 2: DI: Control unit performs the decoding of instruction. It identifies the two important things; first what operation is to be performed and second what addressing modes are used by the instruction. The addressing modes are to be decoded so that the data is brought in the ALU registers for executing the decoded operation. Since decode operation is performed by the control unit, more details related to this operation would be discussed in Unit 11.

Step 3: FoA: In this step the operand address is converted to the direct operand address and is stored in the address part of instruction. This address is used in the next step for instruction execution. In the case of the present instruction format only two possible addressing types – direct and indirect. In the case of direct addressing the address of the operand is already in the operand address part of instruction, thus, no additional micro-operation is needed. However, in case of indirect addressing, the address of the operand is to be fetched using the address portion of the instruction. This fetched address should replace the current address portion of the instruction. This step is shown in Figure 10.5:

When Direct Addressing is used: ... No action is needed.	IR (Address) contains the direct address of the operand.
When Indirect Addressing is used: ... Transfer the operand address portion of instruction to MAR. ... Read the memory using MAR and bring operand address in DR ... Transfer address from DR to IR. Now, IR has a direct address.	T1: MAR ← IR(Address) T2: DR←(MAR) T3: IR(Addrss)← DR(Address)

Figure 10.5: An Indirect cycle

Thus, the IR now contains the direct address of the operand.

Step 4: ExI: The instruction execution is also performed with the help of micro-operations. The first step of instruction execution will require the operand to be brought from the address of main memory to the processor register. This is followed by the arithmetic micro-operation as per the requirements of instruction opcode. The following examples explains the micro-operations required to execute certain instructions.

(1) The step required to execute a simple addition instruction of the form (assuming that indirect bit is set to 0, i.e., it is a direct instruction):

ADD ADR

The following sequence of micro-operations, as given in Figure 10.6, would execute this instruction (after FI cycle):

.. Transfer the ADR in instruction to the MAR. ... Load DR by reading the memory location referred to by ADR. ... Control units then enables addition of AC and DR using ALU. Result is put in AC.	T1: MAR ← IR(ADR) T2: DR ← (MAR) T3: R1 ←R1 + DR

Figure 10.6: Execution cycle of Add instruction

(2) Micro-operations for execution of a conditional jump instruction, which skips the next instruction, if on incrementing the operand results in zero. The incremented value is stored back to the operand location.

INCSKP ADR

The sequence of micro-operations required for this instruction execution are given in Figure 10.7.

Step 1: Bring the operand stored in location ADR to a processor register AC. AC is incremented. ... Transfer the ADR of IR to the MAR. ... Read ADR to DR ... Transfer DR to AC, as increment operation can be performed in AC (assumption). ... Increment the AC. This will set the Flag register.	 T1: MAR ← IR (ADR) T2: DR ← (MAR) T3: AC← DR T4: AC←AC +1
Step 2: Store the AC to ADR ... Transfer the content of AC to DR. ... Store DRinto ADR using MAR and If the content of AC is zero then the next instruction is not to be executed or skipped, to do so increment the value of PC. The control unit checks the zero flag and increments PC if condition is fulfilled.	 T5: DR ← R1 T6: (MAR) ←DR T6: If AC = 0 then PC ←PC + 1

Figure 10.7: Execution cycle of increment and skip instruction

(3) This example shows the sequence of micro-operations required for a branching operation, using a subroutine call instruction. A subroutine call instruction is required to store the return address and then start execution of the subroutine, which will require the change in the value of the PC register. The return address, in general, is stored on a stack, however, for this example, it is assumed that the return address is stored in the subroutine address (ADR) specified in the callinstruction. Assume the following is a subroutine call instruction:

SUBCALL ADR

The sequence of micro-operations to execute this instructionare given in Figure 10.8.

.. Transfer ADR to MAR and the return address, which is in PC is put in the DR. ... To branch to the subroutine, the ADR should be moved to PC. Further, at this address (ADR), the return address is to be put. Please note that ADR is already in MARat time T1. ... The first instruction of the subroutine starts at the ADR plus one, thus, increment PC.	T1: MAR ← IR(ADR) T1: DR ← PC T2: PC ← IR (ADR) T2: (MAR) ←DR T3: PC ←PC + 1

Figure 10.8: Execution cycle of subroutine call instruction

It may be noted that the sequence of micro-operations required to perform instruction execution, viz. FI, DI, FoA and ExI are machine dependent. In this section, we have presented a very simple example for the same.

10.4.3 Interrupt Cycle

Interrupt Processing: In addition to execution of the instruction, the processor should also respond to the interrupt, which may have occurred while instructions of a program are getting executed. It may be noted that programs may have priority and may allow only some of the interrupts, while they are executing. Thus, every computer has provision of enabling interrupts as per their priority.Now, the question is how the processor will check if an enabled interrupt has occurred? One of the solutions to this problem is to keep a control line for interrupt signal and check it after each instruction cycle. Next question is how to acknowledge an interrupt? To acknowledge an interrupt, processor may perform an interrupt cycle, as given in Figure 10.9. The interrupt cycle has been explained assuming that a stack is used to store the address of the instruction, from where the program will be restarted once the interrupt is processed.

.. The return address is in the PC register, which is to be stored on a stack, named STACK, with stack top pointed by stack pointer register (SP). It is being represented as STACK[SP]. DR register is to be used for this operation. ... Increment the stack pointer to next location and start executing the interrupt servicing program by transferring the address of its first instruction, say ISRAddress, to PC.	T1: DR ← PC T2: STACK[SP]←DR T3: SP ←SP+1 T3: PC ←ISRAddress

Figure 10.9: Interrupt cycle using a stack

It may be noted that even interrupt servicing programs are a sequence of instructions. Each of these instructions are executed as per instruction cycle. You may refer to the further readings for more details on instruction cycle.

10.5 INSTRUCTION PIPELINING

In the previous section, you have gone through various steps of instruction execution. Can these steps be performed in parallel to execute an instruction? The answer is NO, as to execute an instruction these steps are to be performed in sequence. However, can the steps of different instruction be performed in parallel to each other or can be executed in an overlapped manner. Execution of several instructions in parallel will require several processing elements, rather breaking the execution of instruction into steps and executing instruction in an overlapped manner may be useful. This is the the principle of instruction pipelining. Thus, a simple instruction pipeline would require to execute instructions in an overlappedway, which will facilitate and reduce the time of

overall instruction execution. This would require that instruction cycle should be divided into equal parts which can be executed in parallel for different stages of instructions. One such decomposition of instruction cycle stages, also called pipeline stages are -fetch the instruction (FI), decode the instruction (DI), fetch operand address(FoA) and execute the instruction (ExI). A new stagehas been added here that allows storage of result back to the memory location, let us call it StR.
Figure 10.10 shows the overlapped execution of seven instructions using a five stage instruction pipelining.

Time Slot	1	2	3	4	5	6	7	8	9	10	11
Instruction 1	FI	DI	FoA	ExI	StR						
Instruction 2		FI	DI	FoA	ExI	StR					
Instruction 3			FI	DI	FoA	ExI	StR				
Instruction 4				FI	DI	FoA	ExI	StR			
Instruction 5					FI	DI	FoA	ExI	StR		
Instruction 6						FI	DI	FoA	ExI	StR	
Instruction 7							FI	DI	FoA	ExI	StR

Figure 10.10: Instruction Pipeline

In Figure 10.10, you may notice that at time slot time slot 5 the pipeline is executing 5 instructions simultaneously, though in different stages. At the end of time slot 5, execution of the first instruction is completed, thereafter, at the end of each time slot a new instruction would be completed or in other words the first instruction gets completed at the end of time slot 5, the second instruction gets completed at the end of time slot 6, the third instruction gets completed at the end of time slot 7 and so on.Thus, the execution of instruction in an overlapped fashion has resulted in almost one instruction execution in one time slot.However, the instruction pipelines suffer from the problem of resource conflicts. For example, in the 5th time slotThe first instruction is storing the result, therefore, would require memory reference; at the same time slot the second instruction is in the execution stage, thus, it will fetch the operand using memory reference and use the ALU to execute this instruction; at the same time the third instruction is in operand fetch stage, which also requires memory reference; the fourth instruction is in the decode stage; and the fifth instruction is in the fetch instruction stage, which also requires memory reference. Thus, the pipelined processor should allow each of these instructions to reference memory simultaneously through different paths so that there is no conflict, otherwise the instructions cannot be executed in the pipeline fashion.

The Problems relating to Pipelined execution:

- Pipelined execution may suffer from resource conflict as explained above.
- The pipelined execution seems good for execution of sequence of instruction, but instructions that require transfer of control, like conditional branch instruction, may cause disruption of pipeline sequence, as the decision whether a branch will be taken or not, can occur only at the execution stage. In case a branch is to be taken then all the subsequent instructions, which were already fetched in the pipelineare to be removed from the pipeline.

Some solutions for problems related to pipelined execution:

The branch penalty can be minimized using any of the following schemes:
- Predicting, if branch will be taken or not and accordingly fetching the next instruction.
- Making provision of pre-fetching those instructions, which may be executed because of a branch.
- Not allowing fetching of the next instruction to pipeline till the branch decision is made.

Check Your Progress 3

1) What is the need of the indirect cycle? Will indirect cycle be needed even if an instruction use register addressing schemes? Justify your answer.

……

……..

……

2) What is fetch cycle? Do the present-day machines also have this cycle?

……

……..

……

3. What is the role of Interrupt cycle?

……

……

……..

10.6 ALU ORGANISATION

An ALU performs simple arithmetic-logic and shift operations. The complexity of an ALU depends on the type of instruction set, which has been realized for it. A simple ALUs can be constructed for performing computation on fixed-point numbers. An ALU for floating-point arithmetic implementation requires more complex control logic and data processing capabilities. Several micro-processor families utilize only fixed-point arithmetic capabilities in the ALUs. For floating point arithmetic or other complex functions, they may utilize an auxiliary special purpose unit. This unit is called arithmetic processors. Let us discuss all these issues in greater detail in this section.

10.6.1 A Simple ALU Organisation

An ALU consists of circuits that perform data processing micro-operations. Figure 10.11 shows the organisationof a fixed point ALU was suggested by John von Neumann in his IAS computer design.

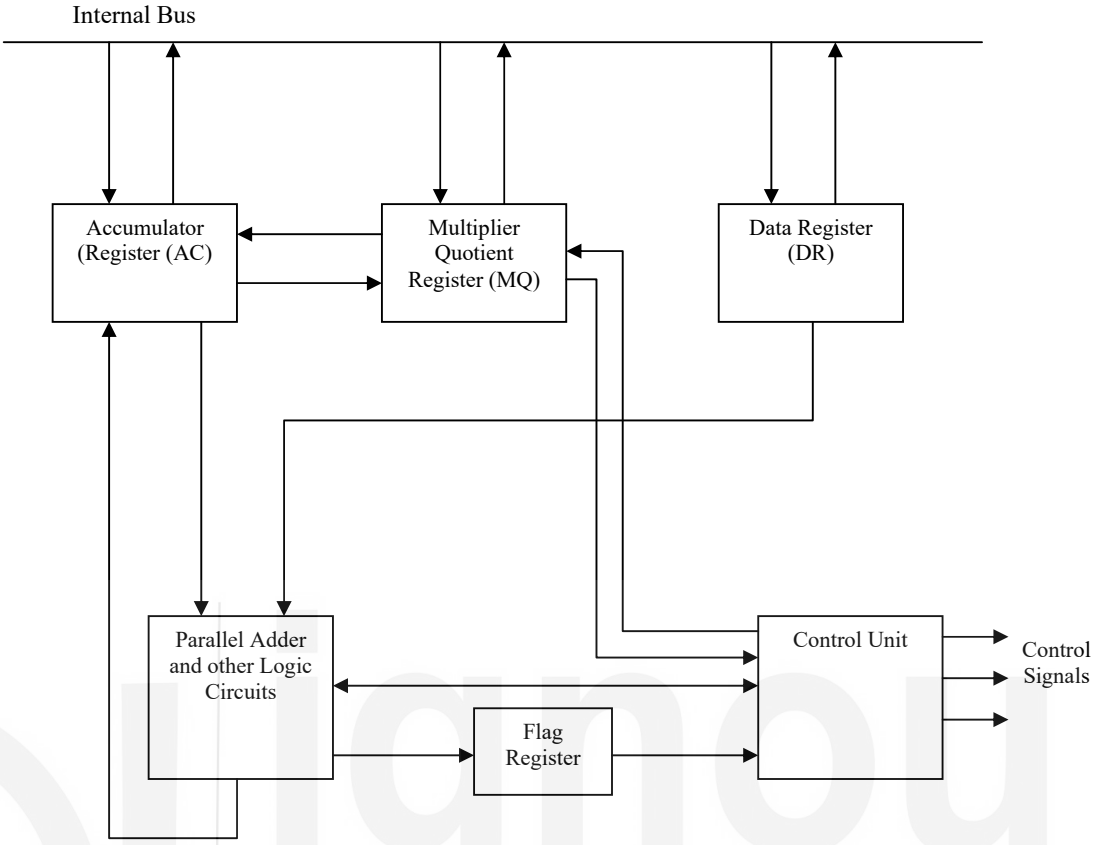


Figure 10.11: Structure of a Fixed-point Arithmetic logic unit

The above structure consists of three registers AC, MQ and DR, with the assumed size of one word each. Please note that the Parallel adders and other logic circuits (these are the arithmetic, logic circuits) this von Neumann machine can have at most two inputs and one output. In other words, it implies that any ALU operation at most can have two input values and will generate single output along with the other status bits. In the Figure 10.11, the two inputs are AC and DR registers, while output is AC register. AC and MQ registers are generally used as a single AC.MQ register. This register is capable of left or right shift operations. Some of the micro-operations that can be defined on this ALU are:

Addition	:	AC ←AC + DR
Subtraction	:	AC ←AC – DR
AND	:	AC ←AC∧ DR
OR	:	AC ←AC ∨DR
Exclusive OR	:	AC ←AC⊕ DR
NOT	:	AC ←AC’

In this ALU organisation multiplication and division were implemented using shift-add/subtract operations. The MQ (Multiplier-Quotient register) is a special register used for implementation of multiplication and division instructions. Please note that in the ALU shown in Figure 10.11, the multiplication and division instructions are not implemented directly using the logic circuits. For more details on these algorithms please refer to further readings. One such algorithm is Booth’s algorithm and you must refer to it in further readings.

For multiplication or division operations DR register stores the multiplicand or divisor respectively. The result of multiplication or division on applying certain algorithm can finally be obtained in AC.MQ register combination. Please note that these are not micro-operations for the given ALU organization, as execution of these two instructions would require a series of shift-add operations.

DR is another important register, which is used for storing second operand. In fact, it acts as a buffer register and stores the data brought from the memory. In machines where we have general purpose registers any of the registers can be utilized as AC, MQ and DR. For more details on ALUs, you can go through the further readings.

10.6.2 A Sample ALU Design

ALU consists of circuits that executes the micro-operations. The data is input to ALU through registers and the output of ALU is also stored in an output register. For performing the input/output of data to ALU, a BUS is used. So, let us first explain how an internal bus can be used for Data transfer.

A computer processor has large number of registers. These registers are used to store data that is required to be processed by the ALU.But, how is the datacommunicated among these registers? One possibility is to create separate data paths from every register to all other registers, however, this connection structure would waste large amount of processor resources. Therefore a shared media call internal BUS. A BUS consists of shared data lines, which are then connected to every register. Using these shared lines any two registers can communicate with each other, at a time. The number of lines in the shared BUS is kept same as the size of registers.

A register is selected for the transfer of data through bus with the help of control signals. The common data transfer path, that is the bus lines, are made using the multiplexercircuit.Figure 10.12 shows an example of 2-bit data bus using 2×1 multiplexers. Please note the size of the registers is also of two bits.
The construction of a bus system for two 2-bit registers using two 2×1 multiplexers is shown in the Figure 10.12. Each register has two bits, viz. Bit 1 and Bit 0. Each multiplexer has 2-bit data input, numbered 0 and 1, and one control or selection lines, C_0. The circuit assumes no enable bits. The 0^{th}data input of MUX 0is connected to the corresponding Bit 0 of Register A and the 1st data input of MUX 0 is connected to Bit 0 of Register B. Similarly, the Bit 1 of Register A and Bit 1 of Register B are connected to 0th data input and 1st data input of MUX 1 respectively. There is just 1 selection line S, when S is 0, then 0th input values for MUX 0 and MUX 1 are transferred to the output, that is the content of Register A is transferred on the bus, whereas, when S is 1 bits of Register B are selected for transmission on the bus.

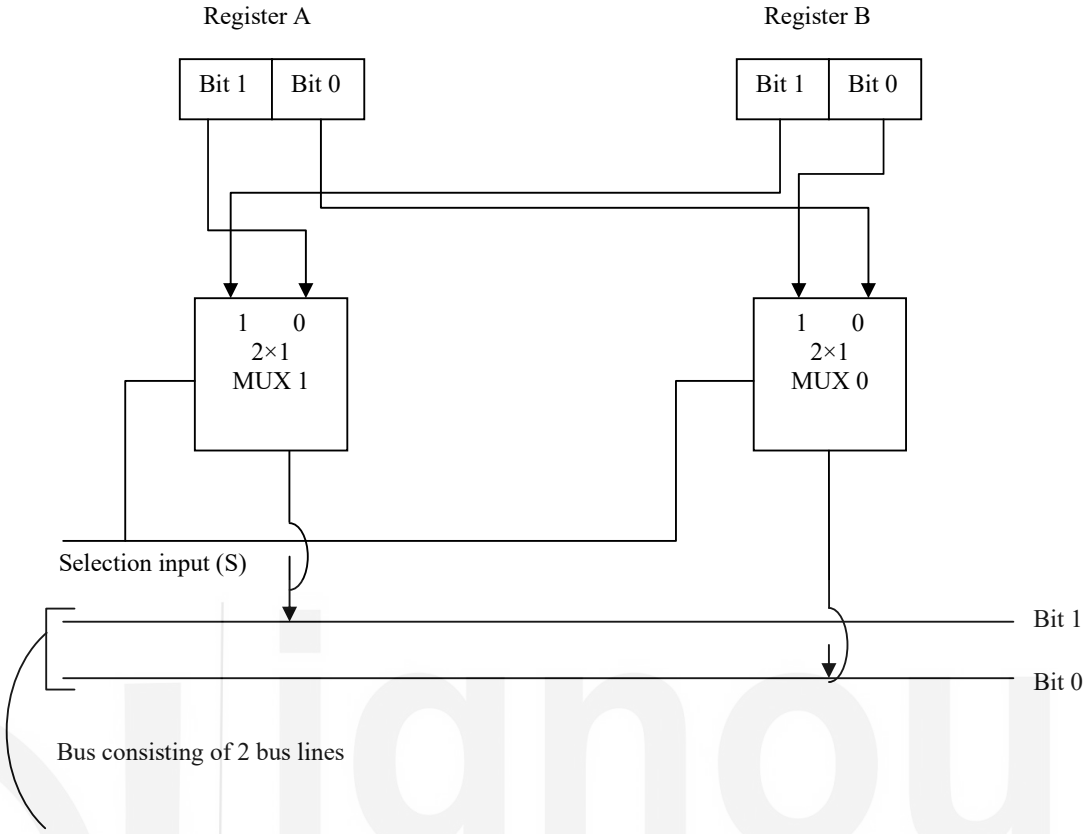


Figure 10.12: Implementation of a 2-bit BUS

The Figure 10.13lists the selection of data based on the selection input to the multiplexers.

S	MUX 1	MUX 0	Comments
0	Bit 1 of Register A	Bit 0 of Register A	Content of Register A is put on the bus lines.
1	Bit 1 of Register B	Bit 0 of Register B	Content of Register B is put on the bus lines.

Figure 10.13: Bus Line Selection

Thus, to construct a bus for 2 registers of 2-bits each, you would require two 2×1 multiplexers. Similarly, to construct a bus for 8 registers of sixteen bits each, you would require sixteen 8×1 multiplexers, which will have 3 selection input. Please note one multiplexer is needed for transfer of one bit. Since sixteen bits are to be transferred, therefore, sixteen multiplexers would be needed. Further, one of the 8 registers would be selected to transfer data on the BUS, therefore, 3 selection input would be needed, as $2^3 = 8$.

Implementation of Arithmetic Circuits for Arithmetic Micro-operation

An arithmetic circuit can be implemented using a number of full adder circuits or parallel adder circuits, one such circuit is shown in Unit 3 of Block 1. Figure 10.14 shows a simple circuit for a 2-bit arithmetic circuit. The circuit is constructed by using 2 full adders and two 2×1 multiplexers, which requires just one selection input. Please recollect that afull adder circuit adds two input bits and one carry-in bit to produce one sum-bit and one carry-out-bit.

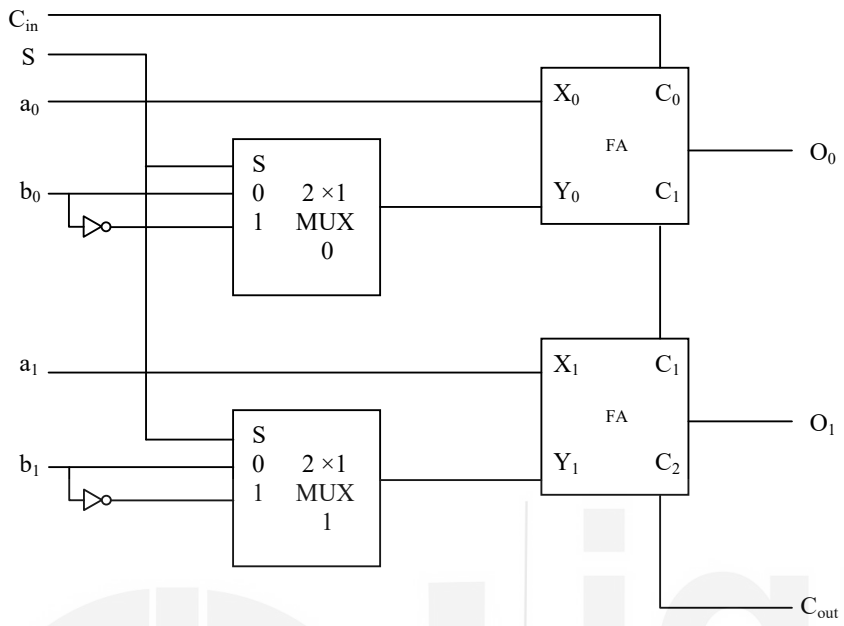


Figure 10.14: A two-bit arithmetic circuit (adder-subtractor)

The multiplexer controls one of the input to the circuit resulting in a set of micro-operations. Let us find out how the multiplexer control lines will change one of the Inputs for Adder circuit. Figure 10.15 shows the two inputs that are possible in the Figure 10.14. (Please note the convention used in this table, viz. uppercase alphabet indicates a 2-bit data word, whereas the lowercase alphabet indicates a bit.)

Control Input	Output of 4 × 1 Multiplexers		Y input to Adder	Comments
S	MUX 0	MUX 1		
0	b_0	b_1	B	The data word B is input to Full Adders
1	Complement of b_0	Complement of b_1	B′	1's complement of B is input to Full Adders

Figure 10.15: Input to full adders using the multiplexers in Figure 10.14

Now let us discuss how by using the carry-in-bit (C_{in}) and these input values, you can obtain various micro-operations.

Input to Circuits

- Register A bits as $a_0$and a_1, are input to X_0 and X_1 bits of the Full Adders (FA).
- Register B bits are input as given in the Figure 10.15to form the Y input to FA.
- Please note that each bit of register A and register B is fed to different full adder unit.
- Please also note that the A input directly goes to adder but B input can be manipulated through the Multiplexers to create different input values, as shown in Figure 10.15. The B input is controlled by the selection line S.
- The input carry C_{in}, which can be equal to 0 or 1, is input to the full adder that adds the least significant bits. The carry out of this full adder is then fed to the

full adder of the next higher bit and so on. The carry out of the most significant bit full adder is the output of the circuit. Logically it is the same as that of addition operation performed by us. We do pass the carry of lower digits addition to higher digits. The following Boolean function represents the output of this adder circuit:

$$O = X + Y + C_{in}$$

Please note that in Figure 10.15, the value of X is a direct input, but the value of Y is input through the multiplexer using the selection input S. In addition, the value of C_{in} is another input. The arithmetic micro-operations that can be implemented using Figure 10.15 are given in Figure 10.16.

S	C_{in}	Y	$O = X+Y+C_{in}$	Equivalent Micro-Operation	Micro-Operation Name
0	0	B	O = A + B	R ← R1 + R2	Add
0	1	B	O = A + B + 1	R ← R1 + R2 + 1	Add with carry
1	0	B′	O = A+B′	R ← R1 + R2′	Subtract with borrow
1	1	B′	O = A+ B′ + 1	R ← R1 + 2's complement of R2	Subtract

Figure 10.16: Arithmetic Micro-operations implemented using Figure 10.15

Let us refer to some of the cases of the Figure 10.16.

When S= 0, input line B is applied directly to the Y inputs of the full adder. Now,

If input carry C_{in}= 0, the output will be O = A + B
If input carry C_{in}= 1, the output will be O = A + B + 1.

If you choose S= 1, thenB′ forms the Y input to the full adder. So,
If C_{in}= 1, then output D = A + B′ + 1. This is called subtractmicro-operation. (Why?)

Reason: Please observe the following example, where A = 0111 and B=0110, then B′=1001. The sum will be calculated as:

```
  0111      (Value of A)
+ 1001      ( Complement of B)
-----------------------------
1 0000 + (ignore the carry out bit and Add Carry in = 1)
= 0001
```

Thus, it is a subtract micro-operation.

If C_{in}= 0, then D = A + B′ . This is called subtract with borrow micro-operation. (Why?). Let us look into the same addition as above:

```
    0111      (Value of A)
    1001      ( Complement of B)
1   0000 + (Carry in =0) = 0000
```

This operation, thus, is equivalent to:

O = A +B′
O = (A – 1) + (B′ + 1)
=> O = (A – 1) + 2's complement of B
=> O = A – (B+1) Thus, is the name subtract with borrow

Two special cases:
When S= 0, C_{in}=1 and Y input is ZERO.

The output O = A + 0 + C_{in} =>O = A + 1

This micro-operation is an increment micro-operation.
When S= 1,C_{in}=0 and input word Y has all 1's.

Output O = A + All (1s) + C_{in} => D = A – 1 (How? Let us explain with the help of the following example).

This is a decrement micro-operation.

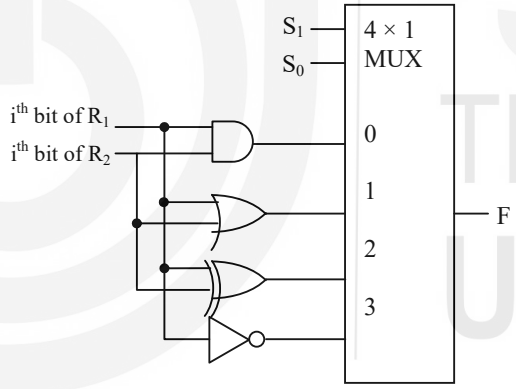
Example: Let us assume that the Register A is of 4 bits and contains the value 0101 and it is added to an all (1) value as:

```
  0101
+ 1111
------
1 0100
------
```

The 1 is carry out and is discarded. Thus, on addition with all (1's) the number has actually got decremented by one.

Implementation of Logic Micro-operations

In many computers only four logic micro-operations, viz. AND, OR, XOR and NOT logicmicro-operations, are implemented. The other logic micro-operations can be derived from these four micro-operations. Figure 10.17 shows one bit, which is the i^{th}bit stage of the four logic operations. Please note that the circuit consists of 4 gates and a 4 × 1 MUX. The ith bits of Register R1 and R2 are passed through the circuit. On the basis of selection inputs S_0 and S_1 the desired micro-operation is obtained.



(a) Logic Diagram

S_1	S_0	Output	The Operation
0	0	$F = R_1 \wedge R_2$	AND Operation
0	1	$F = R_1 \vee R_2$	OR Operation
1	0	$F = R_1 \oplus R_2$	XOR Operation
1	1	$F = R_1'$	Complement of Register R_1

(b) Functional representation

Figure 10.17: Logic diagram of one stage of logic circuit

Arithmetic, Logic and Shift Unit

So, by now we have discussed how the arithmetic and logic micro-operations can be implemented individually. If we combine these two circuits along with shifting logic then we can have a possible simple structure of ALU. In effect ALU is a combinational circuit whose inputs are contents of specific registers. The ALU performs the desired micro-operation as determined by control signals on the input and places the results in an output or destination register. The whole operation of ALU can be performed in a single clock pulse, as it is a combinational circuit. The shift operation can be performed in a separate shift registers but sometimes it can be made as a part of overall ALU. More details on ALU can be studied from the further readings.

10.7 ARITHMETIC PROCESSORS

Arithmetic processorswere needed in the older computer processors to perform arithmetic processing, especially the floating-point arithmetic, as those processorsdid not had the required logic circuits to directly handle such processing. They were also called co-processor, as they were used as an additional processor of some main processor. However, in this era of ultra-large-scaleintegration and beyond, all the fixed point and floating-point computational capabilities are built in the processor. The concept relating to arithmetic processor is explained below.

A typical processor needs most of the control and data processing hardware for implementing non-arithmetic functions. As the hardware costs are directly related to chip area, a floating-point circuit being complex in nature is costly to implement. They need not be included in the instruction set of a processor. In such systems, floating-point operations were implemented by using software routines. This implementation of floating-point arithmetic is definitely slower than the hardware implementation. Now, the question is whether a processor can be constructed only for arithmetic operations. A processor, if devoted exclusively to arithmetic functions, can be used to implement a full range of arithmetic functions in the hardware at a relatively low cost. This can be done in a single Integrated Circuit. Thus, a special purpose arithmetic processor, for performing only the arithmetic operations, can be constructed. This processor physically may be separate yet can be utilized by the processor to execute complex arithmetic instructions. Please note in the absence of arithmetic processors, these instructions may be executed using the slower software routines by the processor itself. Thus, this auxiliary processor enhances the speed of execution of programs having a lot of complex arithmetic computations.

An arithmetic processor also helps in reducing program complexity, as it provides a richer instruction set for a machine. Some of the instructions that can be assigned to arithmetic processors can be related to the addition, subtraction, multiplication, and division of floating-point numbers, exponentiation, logarithms and other trigonometric functions.

How can this arithmetic processor be connected to the CPU?

If an arithmetic processor is treated as one of the Input / Output or peripheral units then it is termed as a peripheral processor. The CPU sends data and instructions to the peripheral processor, which performs the required operations on the data and communicates the results back to the CPU. A peripheral processor has several registers to communicate with the CPU. These registers may be addressed by the CPU as Input /Output register addresses. The CPU and peripheral processors are normally quite independent and communicate with each other by exchange of information using data transfer instructions. This type of connection is called loosely coupled.

If the arithmetic processor has a register and instruction set which can be considered an extension of the CPU registers and instruction set, then it is called a tightly coupled processor. Here the CPU reserves a special subset of code for arithmetic processor. In such a system the instructions meant for arithmetic processor are fetched by CPU and decoded jointly by CPU and the arithmetic processor, and finally executed by arithmetic processor. Thus, these processors can be considered a logical extension of the CPU. Such attached arithmetic processors were termed as co-processors.

These days floating point units are implemented as a part of the processor itself. More details on these can be found in further readings.

☞Check Your Progress 4

1. Explain the implementation of ALU.

……

……

……

Instruction fetch: fetching the

2. What is an Arithmetic Processor?

……

……

……

10.8 SUMMARY

This unit discusses the concept of instruction execution for a hypothetical machine with the help of micro-operations. It also describes very simplified view of implementation of micro-operations using combinational and sequential circuits. The idea is to give you a basic information about the implementation of a computer system based on its instruction set. The unit also discusses the concept of register transfer language for representing the micro-operations. The unit also defined the concept of Instruction Pipeline. The unit also discussed the hardware implementation of micro-operations. The unit shows a simple implementation of bus, which is the backbone for any register transfer operation. This is followed by a discussion on arithmetic circuit and micro-operation there on using full adder circuits. The logic micro-operation implementation has also been discussed. Finally, the unit also discussed the arithmetic processors.
You may refer to the further readings for more details on micro-operation concept and instruction cycle.

10.9 SOLUTIONS / ANSWERS

Check Your Progress 1

1. An address register is used to store memory address or can be used to compute memory address of instructions or operands.
2. To address 20 registers, you may require address field of length 5 bits, as $2^5 = 32$, thus, about 12 addresses are unused.
3. Independent set of conditional codes can help in parallel checking of conditions.
4. Yes. Several operating system allocate memory space for storing such information for later use.

Check Your Progress 2

1. Memory read operation requiresthe address of the location to be read. This address is first applied on the address BUS and the control unit enables memory read. Thus, the content of the addressed location is put in the data BUS. At the same time a data register is enabled to store the data on the data BUS. These operations can be represented as: Address BUS ←MAR ;DR ← Data BUS. The overall operation can be represented as: DR ← (MAR)
   In memory write operationthe memory address, where data is to be written, is applied on address BUS and the data that is to be stored/written is put on the data BUS. At the same time memory write operation is enabled by the control unit. This operation can be represented as: (MAR) ← DR

2. No, as multiplication operation will be implemented using addition and shift micro-operations.
3. (i) AND with R2 containing 0000 0000
   (ii) Initially XOR of R1 with R1 will clear R1, then perform OR with R2 having value 11001000

Check Your Progress 3

1. Indirect cycle is needed, when indirect memory addressing is used by an instruction. It converts an indirect address to a direct address. If an instruction is using register addressing scheme, then indirect cycle may be required only if instruction is using register indirect addressing. The indirect cycle in this case would require simple register transfer micro-operation.
2. Fetch cycle is primarily used for fetching instruction that is to be executed from the memory. The present-day machines may use instruction cache, instruction prefetch buffer etc., yet this instruction is needed to be moved to the processor, which may execute it. Thus, the nature of the fetch cycle may change but it may be required.
3. Interrupt cycle is responsible for acknowledging an interrupt. When an interrupt occurs in a computer, it is acknowledged, when the instruction execution gets completed. The processor checks, if an interrupt has occurred, if it is then an interrupt cycle is performed.

Check Your Progress 4

1. The implementation of ALU can be done with the help of combinational and sequential circuits. The combinational circuits perform the computations. The results of these computations is stored in sequential circuits. The internal register bus is used for input and output to ALU. Arithmetic circuit like adders may be used to perform arithmetic micro-operations, logic gates may be used to perform logic micro-operations and shift registers may be used to perform shift operations.
2. Arithmetic processor performs arithmetic computation. These may be support processors to a computer.

UNIT 11 THE CONTROL UNIT

Structure **Page No.**

11.0 Introduction
11.1 Objectives
11.2 The Control Unit
11.3 The Hardwired Control
11.4 Wilkes Control
11.5 The Micro-Programmed Control
11.6 The Micro-Instructions
11.6.1 Types of Micro-Instructions
11.6.2 Control Memory Organisation
11.6.3 Micro-Instruction Formats
11.7 The Execution of Micro-Program
11.7.1 Micro-instruction Encoding
11.7.2 Micro-instruction Sequencing
11.8 Design Issues of control Unit
11.9 Summary
11.10 Solutions/ Answers

11.0 INTRODUCTION

In the previous units of this block, instruction set architecture and concept of micro-operations were discussed. The micro-operations are used to provide execution environment for the instruction set in a computer system. The micro-operations are implemented as a part of the ALU and processor internal BUS.

The control unit is responsible for issuing the control signals, as per the micro-operationsrequirements of the instructions of a computer. In addition, it controls all the other units of a computer system. In this unit we are going to discuss the functions of a control unit and its implementation mechanisms like hardwired control unit and micro-programmed control unit. The micro-programmed control unit is popular amongst the Intel computer architecture due to its flexibility and legacy requirements. The hardwired control unit and other computer logic circuitrycan be designed using Hardware Description Languages (HDL). The input to these languages are the electronic circuit structure and expected behaviour.Some of the specialized HDLprogramming languages areVHDL, Verilog etc. Discussion on these languages is beyond the scope of this course.
The unit discusses the basic requirements of a control unit, followed by the hardwired control unit andWilkes control unit. Finally, we will discuss the micro-programmed control.

11.1 OBJECTIVES

After going through this unit, you will be able to:

(a) define what is a control unit and its function;
(b) describe a simple control unit organization;
(c) define a hardwired control unit;
(d) define the micro-programmed control unit;
(e) define the term micro-instruction; and
(f) identify types and formats of micro-instruction.

11.2 THE CONTROL UNIT

The processor of a computer consists of three basic components – a set of registers, the arithmetic logic unitand the control unit. All these components are connected through an internal BUS. The role of a control unit is to ensure that an instruction gets executed, as per the sequence of micro-operations for that instruction. This process also involves decoding the instruction that is to be executed. The control unit issues control signals so that these instructions are executed correctly. The basic responsibilities of the control unit are to control:

a) the data exchange of the processor with the memory or I/O modules or interfaces.
b) the internal operations in the processor, which may include:

- moving data between registers using internal BUS or moving data from/to memory location using system BUS (register transfer micro-operations)
- making ALU to perform a particular operation on the data
- regulating other internal operations.

But how does a control unit control the above operations? What are the functional requirements of the control unit? What is its structure? Let us explore answers to these questions in the next sections.

Functional Requirements of Control Unit

Let us first try to define the functions, which a control unit must perform tocause instructions execution. But to define the functions of a control unit, one must know what resources and means it has at its disposal. A control unit must know about the:

(a) Basic components of the processor
(b) Micro-operation this processorcan perform

The processor of a computer consists of the following basic functional components:

- **The Arithmetic Logic Unit (ALU),** which performs the basic arithmetic and logical operations.
- **Registers** which are used for information storage within the processor.
- **Internal Data Paths:** These paths are useful for moving the data between two registers or between a register and ALU.
- **External Data Paths:** The roles of these data paths are normally to link the processor registers with the memory or I/O interfaces. This role is normally fulfilled by the system bus.
- **The Control Unit:** This causes all the operations to happen in the processor.

The micro-operations performed by the processor can be classified as:

- Micro-operations for data transfer from register-register, register-memory, I/O-register etc.
- Micro-operations for performing arithmetic, logic and shift operations. These micro-operations involve use of registers for input and output.

The basic responsibility of the control unit lies in the fact that the control unit must be able to guide various components of processor to perform a specific sequence of micro-operations to achieve the execution of an instruction.

What are the functions, which a control unit performs to make an instruction execution feasible? The instruction execution is achieved by executing micro-operations in a specific sequence. For different instructions this sequence may be different. Thus, the control unit must perform two basic functions:

- Cause the execution of a micro-operation.
- Enable the processor to execute a proper sequence of micro-operations, which is determined by the instruction to be executed.

But how are these two tasks achieved? The control unit generates control signals, which in turn are responsible for achieving the two tasks stated above. But how are these control signals generated? We will answer this question in later sections. First let us discuss a simple structure of control unit.

Structure of Control Unit

A control unit has a set of input values based on which it produces an output control signal, which in turn performs micro-operations. These output signals control the execution of a program. A general model of control unit is shown in Figure 11.1.

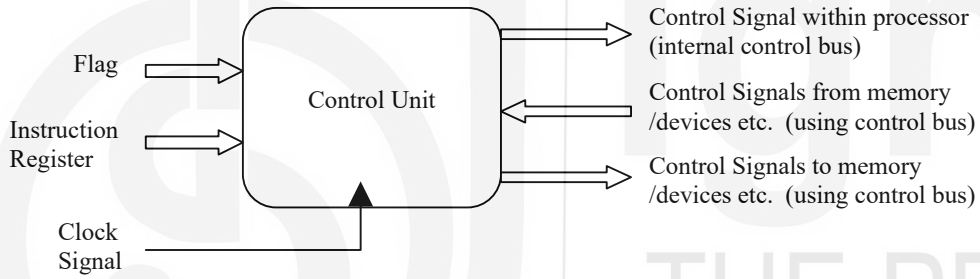


Figure 11.1: A General Model of Control Unit

In the model given above the control unit is a black box, which has certain inputs and outputs.

The inputs to the control unit are:

- **The Master Clock Signal:** This signal causes micro-operations to be performed in a sequence. In a single clock cycle either a single or a set of simultaneous micro-operations can be performed. The time taken in performing a single micro-operation is also termed as processor cycle time or the clock cycle time in some machines.

- **The Instruction Register:** It contains the operation code (opcode) and addressing mode bits of the instruction. It helps in determining various instruction cycles, as given in Unit 10, that are to be performed for a specific instruction and hence determines the related micro-operations.

- **Flags:** Flag represent the conditional codes that can be used in decision making. Flags are set by the ALU operations.For example, a zero flag, if set,communicates to the control unit that the result of last ALU operations was zero. Thus, if processor wasexecuting theISZ instruction(skip the next instruction if zero flag is set), the next instruction should be skipped. This action is initiated by the control unit, whichwouldincrementPC by one program instruction length, thus skipping the next instruction.

- **Control Signals from Control Bus:** Some of the control signals are provided to the control unit through the control bus. These signals are issued from outside the processor. Some of these signals are interrupt signals and acknowledgement signals.

Based on the input signals the control unit activates certain output control signals, which in turn are responsible for execution of an instruction. These output control signals are:

- **Control signals, which are required within the processor:** These control signals cause two types of micro-operations, viz., for data transfer from one register to another; and for performing an arithmetic, logic and shift operation using ALU.

- **Control signals to control bus:** These control signals transfer data from or to processor register to or from memory or I/O interface. These control signals are issued on the control bus to activate a data path on the data / address bus etc.

A control unit must know how all the instructions would be executed. It should also know about the nature of the results and the indication of possible errors. All this is achieved with the help of flags, opcodes, clock andcontrol signals.

A control unit contains a clock portion that provides clock-pulses. This clock signal is used for determining the timing sequence of the micro-operations. In general, the timing signals from control unit are kept sufficiently long to accommodate the propagational delays of signals within the processor along various data paths. Since within the same instruction cycle different control signals are generated at different times for performing different micro-operations, therefore a counter can be utilised with the clock to keep the count. However, at the end of each instruction cycle the counter should be reset to the initial condition. Thus, the clock to the control unit must provide counted timing signals. Examples, of the functionality of control units along with timing diagrams are given in further readings.

How are these control signals applied to realisemicro-operations? *The control signals are applied directly as the binary inputs to the logic gates of the logic circuits that are responsible for implementing micro-operations.* All these inputs are the control signals, which are applied to select a circuit (for example, select or enable input) or a path (for example, multiplexers) or any other operation in the logic circuits.

11.3 THE HARDWIRED CONTROL

In the last section, we have discussed the control unit in terms of its inputs, output and functions. A variety of techniques have been used to organize a control unit. Most of them fall into two major categories:

1. Hardwired control organization
2. Microprogrammed control organization.

In the hardwired organization, the control unit is designed as a combinational circuit or a sequential circuit. In other words, control units is implemented using logic gates, flip-flops, counter circuits, decoder circuit, select and enable input to various logic circuits etc. Since the hardwired control unit is made up of fast logic circuits, it can be optimised for fast operations.
The block diagram of a hardwired control unit is shown in Figure 11.2. The major input to the circuit are instruction register, the clock, and the flags. The control unit uses the opcode of instruction stored in the IR register to perform different actions. The opcode can be used to decode a typical sequence of control signals that are

responsible for execution of that instruction. Selecting separate instruction line for each instruction simplifies the control logic. This control line selection can be performed by a decoder. Decoder are explained in the Unit 3, it uses n input line to select one output line out of 2^n lines.

The clocksignal is input to sequencing logic andforms one of the input of control unit. The sequencing logic issues a repetitive sequence of pulses for the execution of micro-operation(s). These timing signals control the sequence of execution of instruction and determine what control signal needs to be applied at what time for instruction execution. Please note a typical example, of timing sequence for execution of micro-operations of different sub-cyclesof an instruction are given in Unit 10

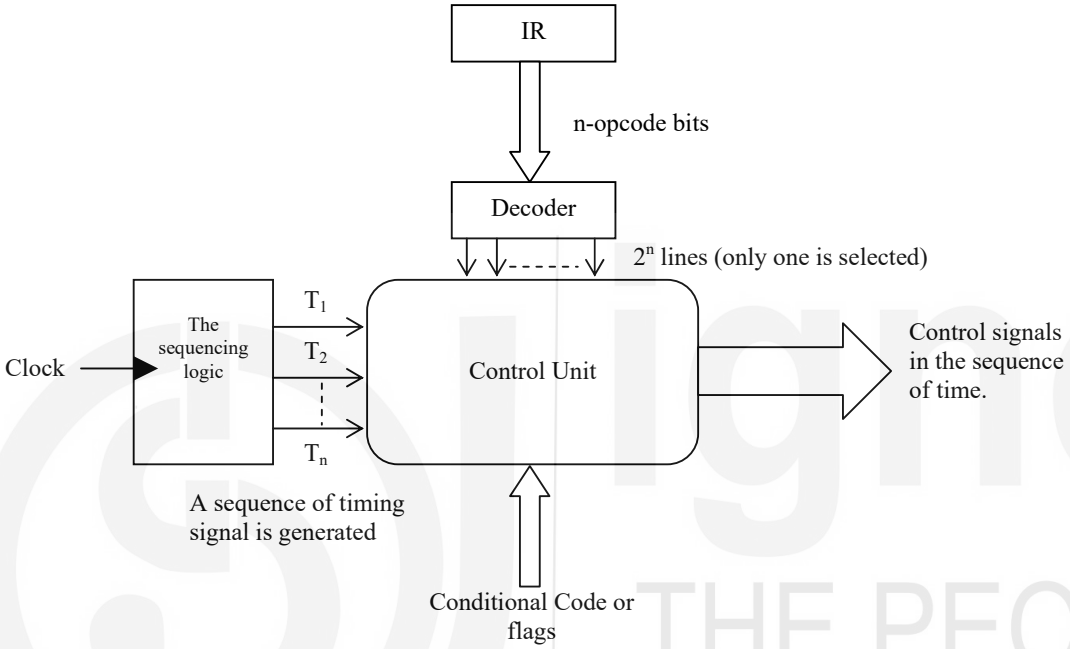


Figure 11.2: Block Diagram of Hardwired Control Unit

☞Check Your Progress 1

1. What are the inputs to control unit?
   .....
   .....
   ……..
2. How does a control unit control the instruction cycle?
   .....
   .....
   ……..
3. What is a hardwired control unit?
   .....
   .....
   ……..

11.4 WILKES CONTROL

Prof. M. V. Wilkes of the Cambridge University Mathematical Laboratory coined the term microprogramming in 1951. He provided a systematic alternative procedure for designing the control unit of a digital computer. During instruction executing a machine instruction, a sequence of data transformations due to arithmetic, logic and shirt micro-operations and transfer of information from one register in the processor due to register transfer micro-operations take place. **Because of the analogy between the execution of individual steps in a machine instruction to the execution of the individual instruction in a program, Wilkes introduced the concept of microprogramming.** The Wilkes control unit replaces the sequential and combinational circuits of hardwired control unit by a simple control unit in conjunction with a storage unit that stores the sequence of steps of instruction that is a micro-program.

In Wilkes microinstruction has two major components:

a) Control field which indicates the control lines which are to be activated and
b) Address field, which provides the address of the next microinstruction to be executed.

Figure 11.3 is an example of Wilkes control unit design.

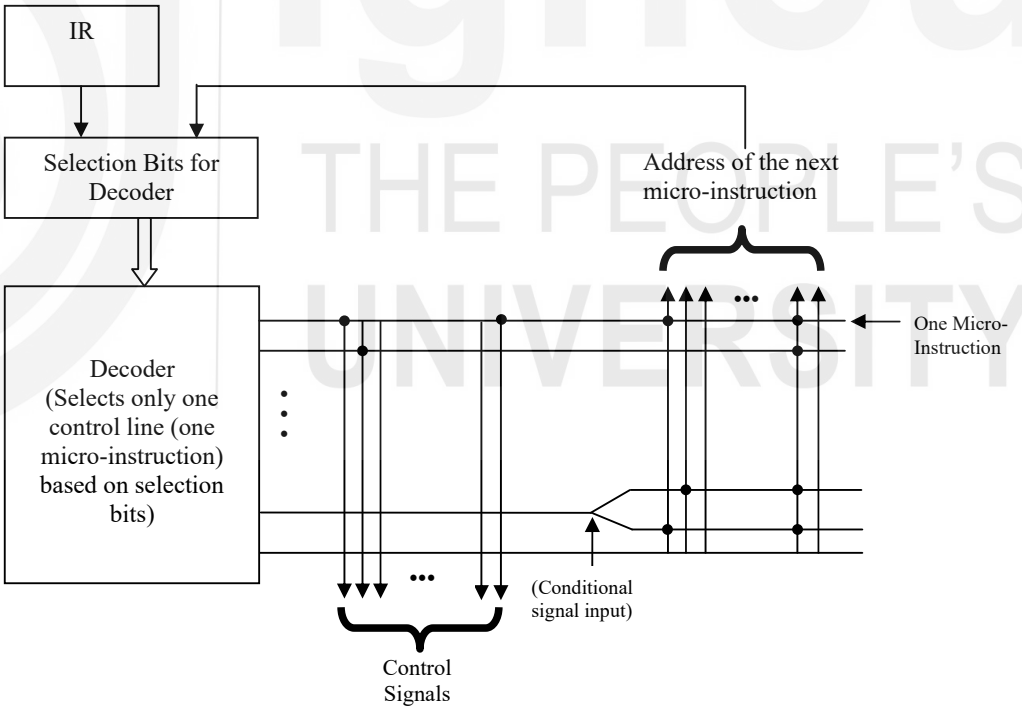


Figure 11.3: Wilkes Control Unit

The control memory in Wilkes control was organized, as a PLA's like matrix made of diodes. Each horizontal line on this matrix consists of two components, viz. the control signals and the address of the next micro-instruction. The next micro-instruction register stores the address of the next micro-instruction to be loaded. Please note that this register should be loaded at the falling edge of clock, that is once the previous micro-instruction completes its execution. The next micro-instruction to be executed is either specified by IR, after the instruction has been decoded or by the address of the next micro-instruction as specified in the micro-instruction itself. The

register input passes through the address decoder and the decoded line of the control matrix is selected for generating the control signals for the processor.The Wilkes control unit also provides handling of conditions. For example, a condition like zero flag can be attached to the conditional signal input, which determines the micro-instruction to be executed next. More details on the Wilkes control unit may be studied from the further readings.

11.5 THE MICRO-PROGRAMMED CONTROL

An alternative to a hardwired control unit is a micro-programmed control unit. Wilke's control unit is one of the examples of micro-program control unit. A micro-program is also called firmware (midway between the hardware and the software). It consists of:

(a) One or more micro-operations to be executed; and
(b) The information about the micro-instruction to be executed next.

The general configuration of a micro-programmed control unit is shown in Figure 11.4.

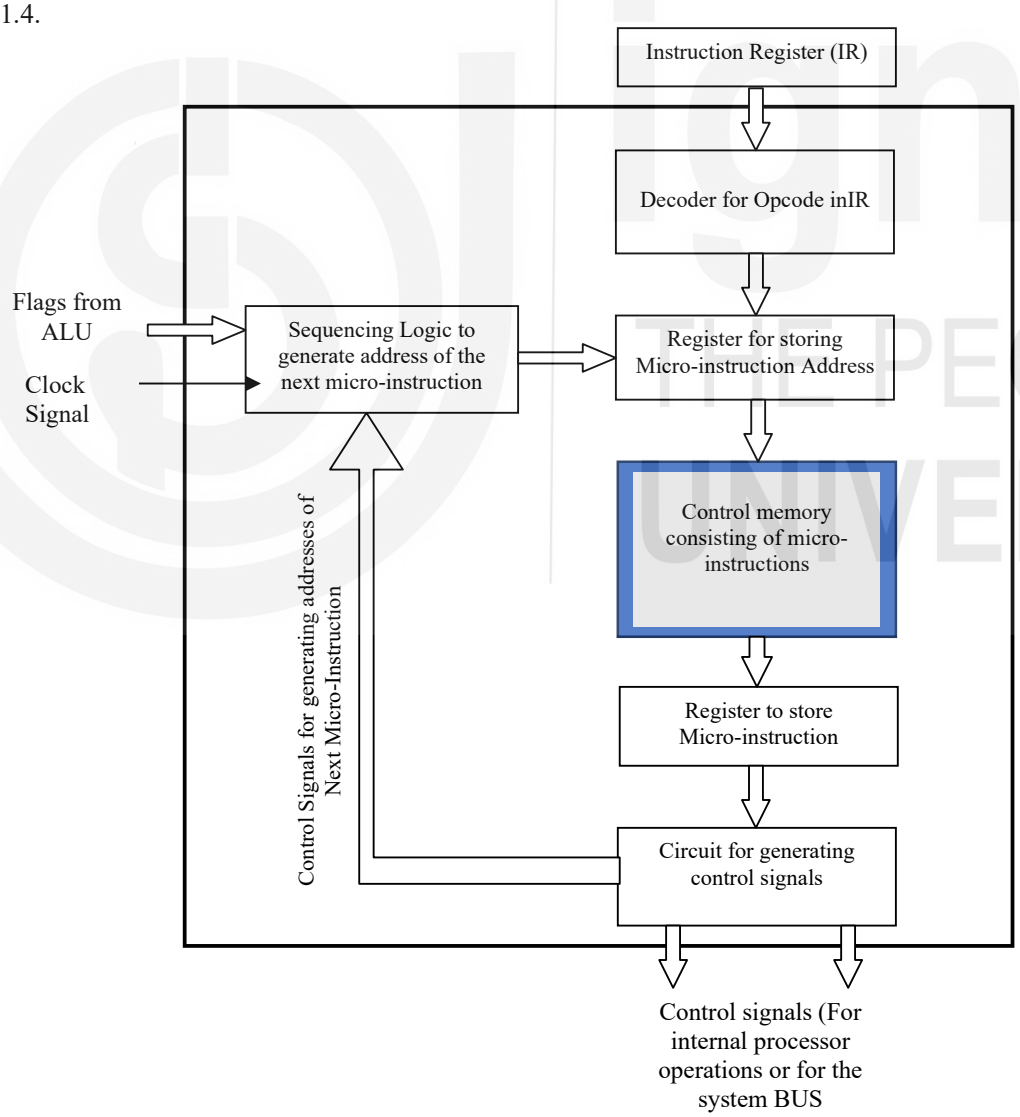


Figure 11.4: Operation of Micro-Programmed Control Unit

The control memory of the micro-programmed control unit stores the micro-instructions. The micro-instruction address register stores the address of the micro-instruction, which should be used to generate the control signal and the address of next micro-instruction.The micro-instruction register stores the last read micro-instruction, which is used to generate the control signals for performing micro-operations and control signals for generating address of the next micro-instruction. . A micro-instruction execution primarily involves the generation of desired control signals and signals used to determine the next micro-instruction to be executed. The sequencing logic of this control unit loads the micro-instruction address register. It also issues a read command to control memory, which stores the micro-instrucitons. The following functions are performed by the micro-programmed control unit:

1. The sequence logic unit specifies the address of the control memory word which contains the micro-instruction that is to be read, in the micro-instruction address register of the Control Memory. It also issues the READ signal to the control memory, so that the desired micro-instruction can be read.
2. The desired control memory word containing the desired micro-instruction is read into the micro-instruction register.
3. The micro-instruction register forms the input to the logic circuit that generates the control signals based on the current micro-instruction. Further, this circuitry also generates the control signals that can be used by sequencing logic to generate the address of micro-instructionin the control memory that is to be executed next.
4. The sequencing logic uses the control signals, as stated above, and flag register to computethe address of the micro-instruction that is to be executed next.

As we have discussed earlier, the execute cycle steps of micro-operations are different for all instructions in addition the addressing mode may be different. All such information generally is stored is coded in the instruction, which is stored in the instruction Register (IR). The IR input to Micro-Instruction Address Register for Control Memory is used for determining the micro-instruction, which performs the execute cycle of the instruction. The decoder after IR uses the IR register to generate the address of the first micro-instruction in control memory for the specified instruction in IR (Refer to Figure 11.4).

☞Check Your Progress 2

1. What is firmware? How is it different from software?
   ……..
2. **State True or False**

   (a) A micro-instruction can initiate only one micro-operation at a time.

   (b) A control word is equal to a memory word.

   (c) Micro-programmed control is faster than hardwired control.

   (d) Wilkes's control does not provide a branching micro-instruction.

3. What will be the control signals and address of the next micro-instruction in the Wilkes control example of Figure 11.3, if the entry address for a machine instruction selects the branching control lineand the conditional bit value for branch is true (assume that out of the two branching lines the bottom line is selected when condition is true)?
   .....

   .....……

   ……..

11.6 THE MICRO-INSTRUCTIONS

A micro-instruction, as defined earlier, is an instruction of a micro-program. It specifies one or more micro-operations, which can be executed simultaneously. On executing a micro-instruction, a set of control signals are generated which in turn cause the desired micro-operation to happen.

11.6.1 Types of Micro-instructions

In general, the micro-instruction can be categorised into two general types. These are branching and non-branching. After execution of a non-branching micro-instruction the next micro-instruction is the one following the current micro-instruction.However, the sequences of micro-instructions are relatively small and last only for 3 or 4 micro-instructions.
A conditional branching micro-instruction tests conditional variable, or a flag generated by an ALU operation. Normally, the branch address is contained in the micro-instruction itself.

11.6.2 Control Memory Organisation

The next important question about the micro-instruction is: how are they organized in the control memory? One of the simplest ways to organize control memory is to arrange micro-instructions for various sub cycles of the machine instruction in the memory. The Figure 11.5 shows such an organisation.

00h	Micro-instructions for fetch cycle	Fetch cycle
01h	···	
02h	···	
03h	Jump to Indirect or Execute Cycle using the addressing mode bits of the instruction	
04h	Micro-instruction for Indirect cycle	Indirect cycle
05h	···	
06h	···	
07h	Jump to Execute cycle	
08h	Micro-instruction for interrupt initiation	Interrupt cycle
09h	···	
0Ah	···	
0Bh	Jump to fetch cycle	
0Ch	Compute address of micro-instruction based on opcode	Execute cycle
0Dh	···	
0Eh	…	
0Fh	Jump to the micro-instruction of opcode	
10h	Micro-instructions for opcode 0 (Let us say LOAD)	Micro-instructions for opcode 0
11h	···	
12h	···	
13h	Jump to fetch or interrupt cycle	
14h	Micro-instruction for opcode 1 (Let us say ADD)	Micro-instructions for opcode 1
15h	···	
16h	···	
17h	Jump to fetch or interrupt cycle	
…	…	…
F8h	Micro-instruction for last opcode (Let us say ISZ)	Micro-instructions for opcode ISZ
F9h	···	
FAh	···	
FBh	···	
FCh	···	
FDh	···	
FEh	···	
FFh	Jump to fetch or interrupt cycle	

Figure 11.5: An example of Control Memory Organisation

Let us give an example of control memory organization. Let us take a machine instruction: Branch on zero. This instruction causes a branch to a specified main memory address in case the result of the last ALU operation is zero, that is, the zero flag is set. The pseudocode of the micro-program for this instruction may be written as:

Test "Zero flag";if SET branch to micro-code at label ZERO
Unconditional branch to micro-code of label NON-ZERO

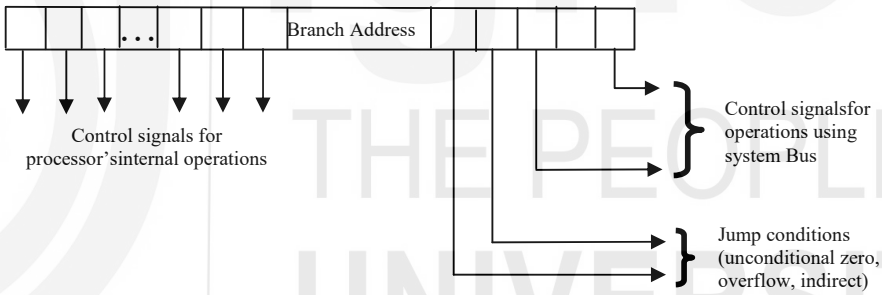
ZERO: Microcode ofsequence of micro-operations required to be executed to replace the PC by the effective address of the instruction operand.

NON-ZERO: *Branch to Interrupt or Fetch cycle.*

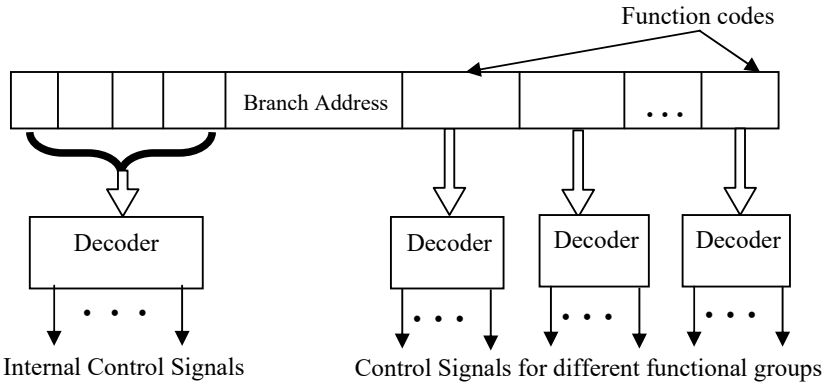
Please note that in case the Zero flag is not SET, then no operation is needed, as next instruction in sequence is to be executed, whose address is already in PC. Thus, next instruction should be fetched.

11.6.3 Micro-instruction Formats

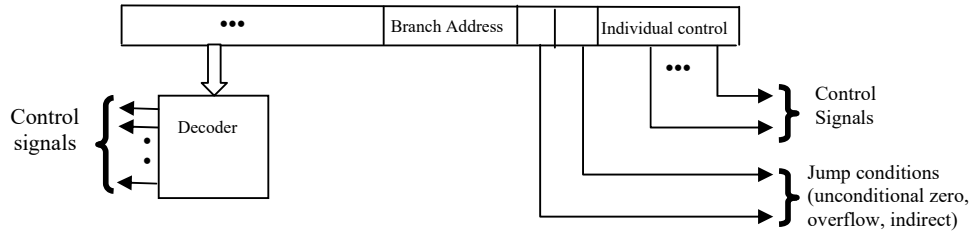
Now let us focus on the format of a micro-instruction. The two widely used formats used for micro-instructions are horizontal and vertical. In the horizontal micro-instruction, each bit of the micro-instruction represents a control signal, which directly controls a single bus line or sometimes a gate in the machine. However, the length of such a micro-instruction may be hundreds of bits. A typical horizontal micro-instruction with its related fields is shown in Figure 11(a).



(a) Horizontal Micro-instruction



(b) Vertical Micro-instruction



(c) A Realistic Micro-instruction

Figure 11.6: Micro-instruction Formats

In a vertical micro-instruction, many similar control signals can be encoded into a few micro-instruction bits. For example, for 16 ALU operations, which may require 16 individual control bits in horizontal micro-instruction, only 4 encoded bits are needed in vertical micro-instruction. Similarly, in a vertical micro-instruction only 3 bits are needed to select one of the eight registers. However, these encoded bits need to be passed from the respective decoders to get the individual control signals. This is shown in Figure 11.6(b).

In general, a horizontal control unit is faster, yet requires wider instruction words, whereas vertical control units, although require a decoder, are shorter in length. Most of the systems use neither purely horizontal nor purely vertical micro-instructions Figure 11.6(c).

11.7 THE EXECUTION OF MICRO-PROGRAM

The micro-instruction cycle can consist of two basic cycles: the fetch and the execute. Here, in the fetch cycle the address of the micro-instruction is generated and this micro-instruction is put in a register used for the address of a micro-instruction for execution. The execution of a micro-instruction simply means generation of control signals. These control signals may drive the processor (internal control signals) or the system bus. The format of micro-instruction and its contents determine the complexity of a logic module, which executes a micro-instruction.These logic module depends on the encoding of micro-instructions, which is discussed next.

11.7.1 Micro-instruction Encoding

One of the key features incorporated in a micro-instruction is the encoding of micro-instructions. What is encoding of micro-instruction? For answering this question let us recall the Wilkes control unit. In Wilkes control unit, each bit of information either generates a control signal or form a bit of next instruction address. Now, let us assume that a machine needs N total number of control signals. If youare using the Wilkes scheme you require N bits for the control signals, one for each control signal in the control unit. In addition, Wilkes control unit also stores the address of the next micro-instruction using address bits.

Since we are dealing with binary control signals, therefore, a ‘N’ bit micro-instruction can represent 2^N combinations of control signals.

The question is do we need all these 2^N combinations?

No, some of these 2^N combinations are not used because:

1. Two sources may be connected by respective control signals to a single destination; however, only one of these sources can be used at a time. Thus, the

combinations where both these control signals are active for the same destination are redundant.

2. A register cannot act as a source and a destination at the same time. Thus, such a combination of control signals is redundant.
3. You can provide only one pattern of control signals at a time to ALU, making some of the combinations redundant.
4. You can provide only one pattern of control signals at a time to the external control bus also.

Therefore, you do not need 2^N combinations. Supposeyou only need 2^K (where K< N) combinations, then you need only K encoded bits instead of N control signals. The K bit micro-instruction is an extreme encoded micro-instruction. Let us touch upon the characteristics of the extreme encoded and unencoded micro-instructions:

Unencoded micro-instructions

- One bit is needed for each control signal; therefore, the number of bits required in a micro-instruction is high.
- It presents a detailed hardware view, as control signal need can be determined.
- Since each of the control signals can be controlled individually, therefore these micro-instructions are difficult to program. However, concurrency can be exploited easily.
- Almost no control logic is needed to decode the instruction as there is one to one mapping of control signals to a bit of micro-instruction. Thus, execution of micro-instruction and hence the micro-program is faster.
- The unencoded micro-instruction aims at optimising the performance of a machine.

Highly Encoded micro-instructions

- The encoded bits needed in micro-instructions are smaller in size than that ofunencoded micro-instructions.
- It provided an aggregated view that is a higher view of the processor as only an encoded sequence can be used for micro-programming.
- The encoding helps in reduction in programming burden; however, the concurrency may not be exploited to the fullest.
- Complex control logic is needed, as decoding is a must. Thus, the execution of a micro-instruction can have propagation delay through gates. Therefore, the execution of micro-program takes longer time than that of an unencoded micro-instruction.
- The highly encoded micro-instructions are aimed at optimizing programming effort.

In general, the micro-programmed control unit designs are neither completely unencoded nor highly encoded. They are slightly coded. This reduces the width of control memory and micro-programming efforts. As shown in Figure 11.7, some of the bits of micro-instructions are unencoded, therefore, can be used to generate the control signals directly. Some of the control bits are organised as fields and can be directly used as input to decoder to generate control signals. Further, a combination of decoded control signals can be passed through a second decoder to generate control signals. These decoding operations are shown in Figure 11.7.

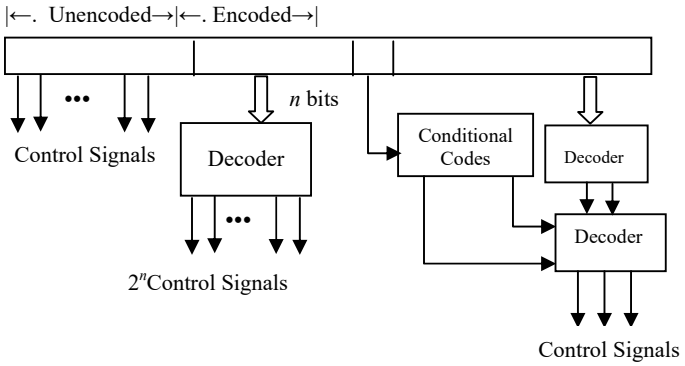


Figure 11.7: Decoding of Micro-instructions

11.7.2 Micro-instruction Sequencing

Another aspect of micro-instruction execution is the micro-instruction sequencing that involves address calculation of the next micro-instruction. In general, the next micro-instruction can be one of the following (refer Figure 11.5):

- Next micro-instruction in sequence
- Calculated on the basis of opcode
- Branch address (conditional or unconditional).

Next instruction in sequence: Figure 11.5 shows one example of control memory. This control organisation has micro-instructions for fetch, indirect and interrupt cycles followed by the execute cycle. You may recall the instruction cycle, as given in Unit 10, the fetchinstruction cycle consists of the following sequence of micro-operations:

T1: MAR ← PC
T2: DR ← (MAR)
T3: PC←PC +1; IR ← DR

One micro-instruction can be created for each timing sequence. For example, for the stated sequence of micro-operations, three micro-instructions, one each for timing T1, T2 and T3 would be created. These micro-instructions would be stored at address 00h, 01h and 02h. The micro-instruction at 03h would be a conditional branch instruction to the indirect or execute cycle. Please also note that at time T3, two micro-operations are to be performed in parallel. Thus, micro-instruction at 02h should generate control signals, which result in the related units of the processor to perform the increment operation on PC and transfer of DR to IR simultaneously.
The micro-instruction at 00h to 03h are to be executed in a sequence to perform the desired operation of instruction fetch.

Branch address (conditional or unconditional): You may please note that the last micro-instruction for instruction fetch is the conditional branch instruction. Please also note that in Figure 11.5, the indirect cycle starts at an address 04h and the execution cycle starts at 0Ch. Thus, the micro-instruction at address 03h would be a conditional branch instruction, having the condition, if the indirect bit is set of not. This conditional branch would be taken to address 0Ch (the start of execution cycle) in case the indirect bit is CLEAR.Thus, if indirect bit is SET, then the next micro-instruction in sequence would be executed, which will be the starting micro-instruction of the indirect cycle that will convert the indirect operand to direct operand. Please also note in the Figure 11.5, that the last micro-instruction of the indirect cycle is an unconditional jump instruction to the execute cycle.

Calculated on the basis of opcode: The opcode of the Instruction Register is used to decode the operation that is to be performed on the operands. The control unit supports this decode operation. In the case of micro-programmed control unit, this opcode can be used to compute the address of the first micro-instruction to be executed to perform the operation. In Figure 11.5, the execute cycle contains the

micro-instructions that perform jump to the micro-instruction address of the desired operation.
We will explain it with the help of an example, assume that in Figure 11.5, the micro-instructions related to opcodes start from micro-instruction address 00h instead of 10h and the micro-instructions of fetch, indirect, interrupt and execute cycles starts at micro-instruction address F0h, F4h, F8h and FCh respectively.Further, we assume that operation of each opcode is performed using just four micro-instructions. Since the control memory has addresses from 00h to FFh, out of which 00h to EFh(a total of F0h) addresses are for storing micro-instructions of various opcodes. Therefore, this control memory can contain micro-instructions for F0h/4h = 3Ch opcodes. Thus, the possible opcodes for such a machine would be $00000000_2$to 00111011_2, or 000000_2 to 111011_2.How these opcodes would be mapped to the related micro-instruction start address. The following table shows these mapping:

Opcode	Starting address of related Micro-instructions	
Binary	Binary	Hexadecimal
0000 00	0000 0000	00
0000 01	0000 0100	04
0000 10	0000 1000	08
0000 11	0000 1000	0C
0001 00	0001 0000	10
0001 01	0001 0100	14
0001 10	0001 1000	18
0001 11	0001 1000	1C
…	…	
1001 00	1001 0000	90
1001 00	1001 0100	94
…	…	
1110 00	1110 0000	E0
1110 01	1110 0100	E4
1110 10	1110 1000	E8
1110 11	1110 1000	EC

Figure 11.8: Mapping of opcode to micro-instruction address

The interesting part of this example is the mapping from the opcode to the micro-instruction address. In Figure 11.8, an opcode is of 6-bit length. To map an opcode to the first related micro-instruction, you just need to append two 0 bits at the least significant position.For example, an opcode 1001 01 will be mapped to a micro-instruction address 1001 0100 or 94h.

Please note different kind of control memory and opcode organisation will make this computational logic a complex one. In any case, you can design the related logic circuit for calculating the micro-instruction address for a given opcode.

You must refer to further readings for more detailed information on Micro-programmed Control Unit Design.

☞Check Your Progress 3

1. **State True or False**

   a) A branch micro-instruction can have only an unconditional jump. ☐

   b) Control store stores opcode-based micro-programs.

   c) A true horizontal micro-instruction requires one bit for every control signal.

d) A decoder is needed to find a branch address in the vertical micro-instruction.

e) One of the responsibilities of sequencing logic (Refer Figure 11.4) is to cause reading of micro-instruction addressed by a micro-program address register. ☐

f) Status bits supplied from ALU to sequencing logic have no role to play with the sequencing of micro-instruction.

2. What art the possibilities for the next instruction address?

.....

.....

.....

.....

.....

3. How many address fields are there in Wilkes Control Unit?

.....

.....

.....

4. Compare and contrast unencoded and highly encoded micro-instructions.

.....

.....

.....

11.8 SUMMARY

In this unit we have discussed the organization of control units. Hardwired, Wilkes and micro-programmed control units are also discussed. The key to such control units are micro-instruction, which can be briefly (that is types and formats) described in this unit. The function of a micro-programmed unit, that is, micro-programmed execution, has also been discussed. The control unit is the key for the optimised performance of a computer. The information given in this unit can be further appended by going through further readings.

11.9 SOLUTIONS/ANSWERS

Check Your Progress 1

1. IR, Timing Signal, Flags Register
2. The control unit issues control signals that cause execution of micro-operations in a pre-determined sequence. This enables execution sequence of an instruction.
3. A logic circuit-based implementation of control unit.

Check Your Progress 2

1. Firmware is basically micro-programs, which are used in a micro-programmed control unit. Firmwareare more difficult to write than software.

2. (a) False (b) False (C) False (d) False

3. Please check the Figure 11.3 from left to right and select the bottom branch line.
   The control signals would be 000…00
   Address of next micro-instruction would be: 100…10

Check Your Progress 3

1. (a) False (b) False (c) True (d) False (e) True (f) False.

2 The address of the next micro-instruction can be one of the following:

- the address of the next micro-instruction in sequence.
- determined by opcode using mapping or any other method.
- branch address supplied on the internal address bus.

3. Wilkes control typically has one address field. However, for a conditional branching micro-instruction, it contains two addresses. The Wilkes control, in fact, is a hardware representation of a micro-programmed control unit.

4.

Unencoded Micro instructions	Highly encoded
• Large number of bits • Difficult to program • No decoding logic • Optimizes machine performances • Detailed hardware view	Relatively less bits Easy to program Need decoding logic Optimizes programming effort Aggregated view

UNIT 12 REDUCED INSTRUCTION SET COMPUTER ARCHITECTURE

Structure **Page No.**

12.0 Introduction
12.1 Objectives
12.2 Introduction to RISC
  12.2.1 Importance of RISC Processors
  12.2.2 Reasons for Increased Complexity
  12.2.3 High Level Language Program Characteristics
12.3 RISC Architecture
12.4 The Use of Large Register File
12.5 Comments on RISC
12.6 RISC Pipelining
12.7 Summary
12.8 Solutions/ Answers

12.0 INTRODUCTION

In the previous units, we have discussed the instruction set, register organization and pipelining, and control unit organization. The trend of those years was to have a large instruction set, a large number of addressing modes and about 16 –32 registers. However, their existed a pool of thought which was in favour of having simplicity in instruction set. This logic was mainly based on the type of the programs, which were being written for various machines. This led to the development of a new type of computers called Reduced Instruction Set Computer (RISC). In this unit, we will discuss about the RISC machines. Our emphasis will be on discussing the basic principles of RISC and its pipeline. You may refer to further readings for more details on this architecture.

12.1 OBJECTIVES

After going through this unit, you should be able to:

- define the reason of increasing complexity of instructions;
- explain the reasons for developing RISC;
- define the basic principles of RISC;
- describe the importance of having large register file;
- describe RISC pipelining.

12.2 INTRODUCTION TO RISC

Reduced Instruction set computer architectures, were initially designed to reduce the complexity of the instruction sets, which included very large number of instructions. The purpose was to design an instruction set that could be designed for better performance with no additional cost of the processor. In fact, the aim of computer processor chip architects has been to design processor chips, which are more powerful than their predecessors, yet are not expensive. Thus, the processor chip designer would like to:

- Optimise the hardware manufacturing cost.
- Optimises the cost for programming scalable/ portable architectures that require low costs for debugging the initial hardware and subsequent programs.

If you review the history of computer families, you will find that the most common architectural change is the trend towards even more complex machines.

12.2.1 Importance of RISC Processors

Reduced Instruction Set Computers recognise a relatively limited number of instructions in comparison to the complex instruction set computers. In addition, a RISC processor also has a limited number of addressing modes with most of the instruction using the register operands. Thus, the instructions of RISC processor are simpler, therefore, can be executed faster. Another advantage is that RISC chips cheaper to design and produce.

In general, an instruction on a RISC machine can be executed in one processor cycle, as RISC use instruction pipeline. As discussed in Unit 11, an instruction pipeline enhances the speed of instruction execution. In addition, the control unit of the RISC processor is simpler and smaller than Complex Instruction Set Computers (CISC). This saved space can be used for building additional registers in the processor. This further enhances the processing capabilities of the RISC processor. Most RISC processor uses the register operands, this necessitates that the memory to register "LOAD" and "STORE" are created as separate independent instructions, which use memory reference operation..

Various RISC Processors

RISC has fewer design bugs; its simple instructions reduce design time. Thus, because of all the above important reasons RISC processors have become popular. Some of the uses if RISC processors are in the mobile processors, desktops, workstation and embedded devices. One of the open instructions set architecture for RISC instruction set is RISC V. This architecture uses the principles of RISC, which are discussed in this unit. For more details, you may refer to the further readings.

12.2.2 Reasons for Increased Complexity

The complexity of computer chips kept growing with the advancement in technology. In this section, we discuss the reasons for increased complexity of instruction set of computers.

Speed of Memory Versus Speed of CPU

In the past, there existed a large gap between the speed of a processor and memory. Thus, an execution of instruction using a program, for example floating point addition, may have to follow a lengthy instruction sequence. The question is; if we make it a machine instruction then only one instruction fetch will be required and rest will be done with control unit sequences. Thus, a "higher level" instruction can be added to machines in an attempt to improve performance.

However, this assumption is not very valid in the present era where the Main memory is supported with Cache technology. Cache memories have reduced the difference between the CPU and the memory speed and, therefore, an instruction execution through a subroutine step may not be that difficult. Let us explain it with the help of an example:

Suppose the floating-point operation ADD A, B requires the following steps (assuming the machine does not have floating point registers) and the registers being used for exponent are E1, E2, and EO (output); for mantissa M1, M2 and MO (output):

- Load the exponent of A in E1

- Load the mantissa of A in M1
- Load the exponent of B in E2
- Load the mantissa of B in M2
- Compare E1 and E2
  If E1 = E2 then MO ← M1 + M2 and EO ← E1
  - Normalise MO and adjust EO
  - Result will be contained in MO, E1

  else if E1 < E2 then find the difference = E2 – E1
  - Shift Right M1, by difference
  - MO ← M1 + M2 and EO ← E2
  - Normalise MO and adjust EO
  - Result is contained in MO, EO

  else E2 < E1, if so find the difference = E1 – E2
  - Shift Right M2 by difference above
  - MO ← M1 + M2 and EO ← E1
  - Normalise MO and adjust E1 into EO
  - Result is contained in MO, EO
- Move the mantissa and exponent of the results to A
- Checks overflow underflow if any.

If all these steps are coded as one machine instruction, then this simple instruction will require several instruction cycles. If this instruction is made a part of the machine instruction set architecture as an instruction: ADDF A, B (Add floating point numbers A and B and store results in A), then it will just be a single machine instruction. All the above steps required will then be coded with the help of micro-operations in the form of Control Unit Micro-Program. Thus, just one instruction cycle (although a long one) may be needed. This cycle will require just one instruction fetch. Whereas in the program memory instructions will be fetched.

However, faster cache memory for Instruction and data stored in registers can create an almost similar instruction execution environment. Pipelining can further enhance such speed. Thus, creating an instruction as above may not result in faster execution.

Microcode and VLSI Technology

It is considered that the control unit of a computer be constructed using two ways; create micro-program that execute micro-instructions or build circuits for each instruction execution. Micro-programmed control allows the implementation of complex architectures more cost effective than hardwired control as the cost to expand an instruction set is very small, only a few more micro-instructions for the control store. Thus, it may be reasoned that moving subroutines like string editing, integer to floating point number conversion and mathematical evaluations such as polynomial evaluation to control unit micro-program is more cost effective. However, such a mechanism may result in slightly slower execution of commonly used instructions.

Code Density and Smaller Faster Programs

The memory was very expensive in the older computer. Thus, there was a need of less memory utilization, that is, it was cost effective to have smaller compact programs. Thus, more complex instruction sets were designed, so that programs are smaller. However, increased complexity of instruction sets had resulted in instruction sets and addressing modes requiring more bits to represent them. It is stated that the code compaction is important, but the cost of 10 percent more memory is often far less than the cost of reducing code by 10 percent out of the CPU architecture innovations.

The smaller programs are advantageous because they require smaller RAM space. Fewer instructions mean fewer instruction bytes to be fetched. But this does not ensures that program written for CISC machines would be smaller in size than that of

programs written for RISC machine. It may be possible that a CISC program is smaller in number of instructions, yet the overall size, in terms of number of bytes, may not be small. This may result from the reason that in RISC we use register addressing and less instruction, which require fewer bits in general. In addition, you may please note that even the compliers on CISC machine favours simpler instructions. Let us explain this with the help of the following example:

Assume a CISC machine has a 4GB byte addressable RAM (2^{32}) and 32 registers (2^5). A machine instruction consists of two operands, one of which should be a register operand. Almost similar RISC machine having the same size of RAM and active registers. Further, the CISC machine uses 16 bit to represent opcode and addressing modes and the RISC machine uses 8 bit to represent opcode and addressing modes. Figure 12.1(a) shows ADD and MOV instructions for the CISC machine. On the other hand, RISC machine would have at least two instruction formats (First to load the data from RAM to register or store the register to memory; or addition operation on register. Figure 12.2 (b) shows these two instruction formats for RISC machine.

16	5	32	
ADD	R1	A	; R1←R1+[A]
ADD	A	R1	; [A]←R1+[A]
MOV	R1	A	; R1← [A]
MOV	A	R1	; [A]←R1

(a) Sample instructions of CISC

8	5	32	
LDA	R1	A	; R1← [A]
STR	R1	A	; [A] ←R1

8	5	5	
ADD	R1	R2	; R1← R1+R2

(b) A sample Load, Store and ADD instruction of RISC

Figure 12.1: Sample machine instructions

Figure 12.1 shows the instructions for a CISC and RISC machine. The size of CISC ADD instruction is 53 bits, therefore, it will be stored in 7 bytes or 56 bits. The load (LDA) and store (STR) instructions of RISC machines are 45 bits long, so would be stored in 6 bytes. In addition, in RISC machine the ADD instruction would use 18 bits or 3 bytes. Consider the following sequence of operations on these two machines:

C=A+B

16	5	32	
MOV	R1	A	; R1← [A]
ADD	R1	B	; R1←R1+[A]
MOV	C	R1	; [C]← R1

Program segment size = 7 × 3 = 21 Bytes

Size in bits = 53 × 3 = 159 bits

(a) Segment to compute C=A+B using sample CISC ISA

8	5	32	
LDA	R1	A	; R1← [A]
LDA	R2	B	; R2← [B]
ADD	R1	R2	; R1←R1+R2 (18 bits)
STR	R1	C	; [C] ←R1

Program segment size = 6 × 3 + 3= 21 Bytes

Size in bits = 45 × 3 + 18 = 153 bits

(b) Segment to compute C=A+B using sample RISC ISA

Figure 12.2: Execution of C=A+B on hypothetical machines

So, the expectation that a CISC will produce smaller programs may not be correct. In addition, in the present time memory is inexpensive, this potential advantage of smaller programs is not so compelling these days.

Support for High-Level Language

With the increasing use of more and higher-level languages, manufacturers had provided more powerful instructions to support them. It was argued that a stronger instruction set would reduce the software crisis and would simplify the compilers. Another important reason for such a movement was the desire to improve performance.

However, even though the instructions that were closer to the high-level languages were implemented in Complex Instruction Set Computers (CISCs), still it was hard to exploit these instructions since the compilers were needed to find those conditions that exactly fit those constructs. In addition, the task of optimising the generated code to minimise code size, reduce instruction execution count, and enhance pipelining is much more difficult with such a complex instruction set.

Another motivation for increasingly complex instruction sets was that the complex HLL operation would execute more quickly as a single machine instruction rather than as a series of more primitive instructions. However, because of the bias of programmers towards the use of simpler instructions, it may turn out otherwise. CISC makes the more complex control unit with larger microprogram control store to accommodate a richer instruction set. This increases the execution time for simpler instructions.

Thus, it is far from clear that the trend to complex instruction sets is appropriate. This has led a number of groups to pursue the opposite path.

12.2.3 High Level Language Program Characteristics

The new architectures should support high-level language programming. A high-level language system can be implemented mostly by hardware or mostly by software, provided the system hides any lower-level details from the programmer. Thus, a cost-effective system can be built by deciding what pieces of the system should be in hardware and what pieces in software.

To ascertain the above, it may be a good idea to find program characteristics on general computers. Some of the basic findings about the program characteristics were:

Variables	Operations	Procedure Calls
Integral Constants 15-25% Scalar Variables 50-60% Array/ structure 20-30%	Simple assignment 35-45% Looping 2-6% Procedure call 10-15% IF 35-45% GOTO FEW Others 1-5%	Most time-consuming operation. FACTS: Most of the procedures are called with fewer than 6 arguments. Most of these have fewer than 6 local variables

Figure 12.3: Typical Program Characteristics

Observations

- Integer constants appeared almost as frequently as arrays or structures.
- Most of the scalars were found to be local variables, whereas most of the arrays or structures were global variables.
- Most of the dynamically called procedures pass lower than six arguments.
- The numbers of scalar variables are less than six.
- A good machine design should attempt to optimize the performance of most time-consuming features of high-level programs.
- Performance can be improved by more register references rather than having more memory references.
- There should be an optimized instructional pipeline such that any change in flow of execution is taken care of.

The Origin of RISC

In the 1980s, a new philosophy evolved having optimizing compilers that could be used to compile "normal" programming languages down to instructions that were as simple as equivalent micro-operations in a large virtual address space. This made the instruction cycle time as fast as the technology would permit. These machines should have simple instructions such that it can harness the potential of simple instruction execution in one cycle – thus, having reduced instruction sets – hence the reduced instruction set computers (RISCs).

Check Your Progress 1

1. List the reasons of increased complexity.

   .....

   .....

   .....

2. State True or False

	T	F
a) The instruction cycle time for RISC is equivalent to CISC.		
b) CISC yields smaller programs than RISC, which improves its performance; therefore, it is very superior to RISC.		
c) CISC emphasizes optional use of register while RISC does not.		

12.3 RISC ARCHITECTURE

Let us first list some important considerations of RISC architecture:

1. The RISC functions are kept simple unless there is a very good reason to do otherwise. A new operation that increases execution time of an instruction by 10 per cent can be added only if it reduces the size of the code by at least 10 per cent. Even greater reductions might be necessary if the extra modification necessitates a change in design.
2. Micro-instructions stored in the control unit cannot be faster than simple instructions, as the cache is built from the same technology as writable control unit store, a simple instruction may be executed at the same speed as that of a micro-instruction.
3. Microcode is not magic. Moving software into microcode does not make it better; it just makes it harder to change. The runtime library of RISC has all the characteristics of functions in microcode, except that it is easier to change.

4. Simple decoding and pipelined execution are more important than program size. Pipelined execution gives a peak performance of one instruction every step. The longest step determines the performance rate of the pipelined machine, so ideally each pipeline step should take the same amount of time.
5. Compiler should simplify instructions rather than generate complex instructions. RISC compilers try to remove as much work as possible during compile time so that simple instructions can be used. For example, RISC compilers try to keep operands in registers so that simple register-to-register instructions can be used. RISC compilers keep operands that will be reused in registers, rather than repeating a memory access or a calculation. They use LOADs and STOREs to access memory so that operands are not implicitly discarded after being fetched. (Refer to Figure 12.1(b) for a simple illustration).

Thus, the RISC were designed having the following:

- **One instruction per cycle**: A machine cycle is the time taken to fetch two operands from registers, perform the ALU operation on them and store the result in a register. Thus, RISC instruction execution takes about the same time as the micro-instructions on CISC machines. With such simple instruction execution rather than micro-instructions, it can use fast logic circuits for control unit, thus increasing the execution efficiency further.

- **Register-to-register operands:** In RISC machines the operation that access memories are LOAD and STORE. All other operands are kept in registers. This design feature simplifies the instruction set and, therefore, simplifies the control unit. For example, a RISC instruction set may include only one or two ADD instructions (e.g. integer add and add with carry); on the other hand a CISC machine can have 25 add instructions involving different addressing modes. Another benefit is that RISC encourages the optimization of register use, so that frequently used operands remain in registers.

- **Simple addressing modes:** Another characteristic is the use of simple addressing modes. The RISC machines use simple register addressing having displacement and PC relative modes. More complex modes are synthesized in software from these simple ones. Again, this feature also simplifies the instruction set and the control unit.

- **Simple instruction formats:** RISC uses simple instruction formats. Generally, only one or a few instruction formats are used. In such machines the instruction length is fixed and aligned on word boundaries. In addition, the field locations can also be fixed. Such an instruction format has a number of benefits. With fixed fields, opcode decoding and register operand accessing can occur in parallel. Such a design has many advantages. These are:

    - It simplifies the control unit
    - Simple fetching as memory words of equal size are to be fetched
    - Instructions are not across page boundaries.

Thus, RISC is potentially a very strong architecture. It has high performance potential and can support VLSI implementation. Let us discuss these points in more detail.

- **Performance using optimizing compilers**: As the instructions are simple the compilers can be developed for efficient code organization also maximizing register utilization etc. Sometimes even the part of the complex instruction can be executed during the compile time.
- **High performance of Instruction execution:** While mapping of HLL to machine instruction the compiler favours relatively simple instructions. In addition, the control unit design is simple and it uses little or no micro-

instructions, thus could execute simple instructions faster than a comparable CISC. Simple instructions support better possibilities of using instruction pipelining.

- **VLSI Implementation of Control Unit:** A major potential benefit of RISC is the VLSI implementation of microprocessor. The VLSI Technology has reduced the delays of transfer of information among CPU components that resulted in a microprocessor. The delays across chips are higher than delay within a chip; thus, it may be a good idea to have the rare functions built on a separate chip. RISC chips are designed with this consideration. In general, a typical microprocessor dedicates about half of its area to the control store in a micro-programmed control unit. The RISC chip devotes only about 6% of its area to the control unit. Another related issue is the time taken to design and implement a processor. A VLSI processor is difficult to develop, as the designer must perform circuit design, layout, and modeling at the device level. With reduced instruction set architecture, this processor is far easier to build.

12.4 THE USE OF LARGE REGISTER FILE

In general, the register storage is faster than the main memory and the cache. Also the register addressing uses much shorter addresses than the addresses for main memory and the cache. However, the numbers of registers in a machine are less as generally the same chip contains the ALU and control unit. Thus, a strategy is needed that will optimize the register use and, thus, allow the most frequently accessed operands to be kept in registers in order to minimize register-memory operations.

Such optimisation can either be entrusted to an optimising complier, which requires techniques for program analysis; or we can follow some hardware related techniques. The hardware approach will require the use of more registers so that more variables can be held in registers for longer periods of time. This technique is used in RISC machines.

It may seem that a large number of registers would lead to fewer memory accesses, however in general, about 32 registers were considered optimum. So how does this large register file further optimize the program execution?

Since most operand references are to local variables of a function in C they are the obvious choice for storing in registers. Some registers can also be used for global variables. However, the problem here is that the program follows function call - return so the local variables are related to most recent local function, in addition this call - return expects saving the context of calling program and return address. This also requires parameter passing on call. On return, from a call the variables of the calling program must be restored and the results must be passed back to the calling program.

RISC register file provides a support for such call-returns with the help of register windows. Register files are broken into an overlapping set of smaller group of registers, as shown in Figure 12.4. Each of these register set can be used for different function/subroutine. A function call automatically changes the set being used. The use from one fixed size window of registers to another, rather than saving registers in memory as done in CISC. Windows for adjacent procedures are overlapped. This feature allows parameter passing without moving the variables at all. The following figure tries to explain this concept:

Assumptions:

Register file contains 138 registers. Let them be called by register number 0 – 137. Further, a program has three functions, viz. main, sorting and Xchange. The operating

system calls function main (f_{main}) which calls function sorting ($f_{sorting}$) and function sorting calls function Xchage (f_{Xchage}).

Registers Nos.	Used for			
0 – 9	Global variables required by f_{main}, $f_{sorting}$, and f_{Xchage}	Function main	Function sorting	Function Xchage
10 – 83	Unused			
84 – 89 (6 Registers)	Used by parameters of f_{Xchage} that may be passed to next call			Temporary variables of function Xchage
90 – 99 (10 Registers)	Used for local variable of f_{Xchage}			Local variables of function Xchage
100 – 105 (6 Registers)	Used by parameters that were passed from $f_{sorting} \rightarrow f_{Xchage}$		Temporary variables of function sorting	Parameters of function Xchage
106 – 115 (10 Registers)	Local variables of $f_{sorting}$		Local variables of function sorting	
116 – 121 (6 Registers)	Parameters that were passed from f_{main} to $f_{sorting}$	Temporary variables of function main	Parameters of function sorting	
122 – 131 (10 Registers)	Local variable of f_{main}	Local variables of function main		
132 – 138 (6 Registers)	Parameter passed to f_{main}	Parameters of function main		

Figure 12.4: Use of three Overlapped Register Windows

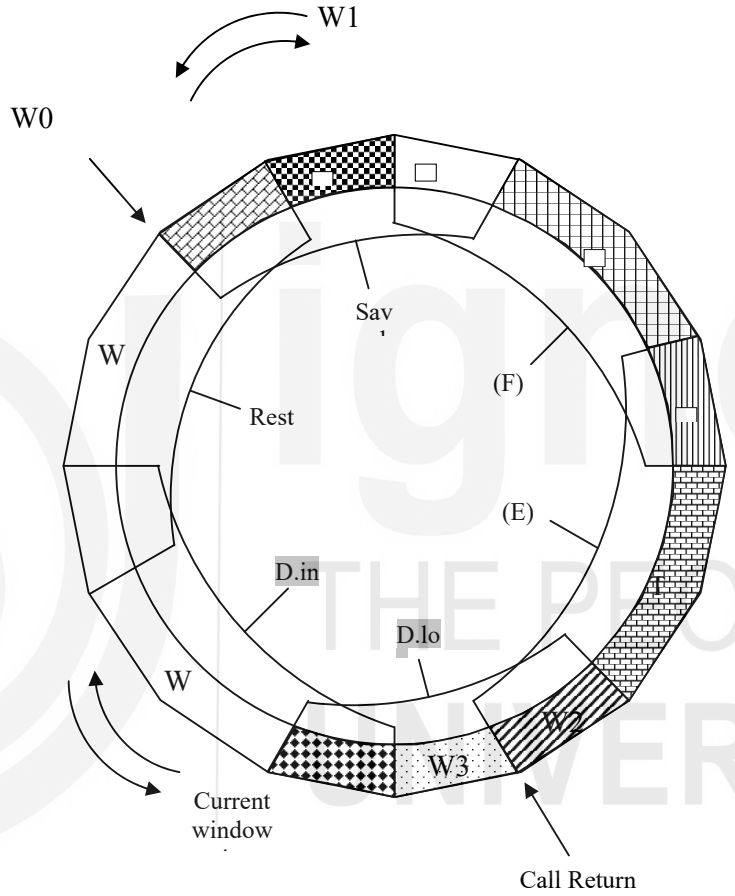
Functioning of the registers: at any point of time the global registers and one set of register being used for a specific function would be active for execution of the program. Thus, for programming purpose there may be only 32 registers. Window in the above example although has a total of 138 registers. This window consists of:

- Global registers which are shareable by all functions.
- Parameters registers for holding parameters passed from the previous function to the current function. They also hold the results that are to be passed back.
- Local registers that hold the local variables, as assigned by the compiler.
- Temporary registers: The temporary registers are used to pass parameters to the function that is called by the presently executing function. The parameters to passed are stored in these temporary registers and passed as parameters to the function that is called by the presently executing function.

But what is the maximum function calls nesting can be allowed through RISC? Let us describe it with the help of a circular buffer diagram, technically the registers as above have to be circular in the call return hierarchy.

This organization is shown in the following figure. The register buffer is filled as function A called function B, function B called function C, function C called function

D. The function D is the current function. The current window pointer (CWP) points to the register window of the most recent function (function D in this case). Any register references by a machine instruction is added with the contents of CWP to compute the address of the registers holding the operands for the executing function. The other register, i.e., the saved window pointer registers points to the most recent register window that has be saved in the memory of the computer. This action will be needed if a further call is made and there is no space for that call. If function D now calls function E arguments for function E are placed in D's temporary registers indicated by D temp and the CWP is advanced by one window.



A.in:	Input register parameters/argument of function A
A.loc:	Local variables of function A
B.in or A.temp	Parameters with which function B is to be called B.in It is same as A.temp which are parameters passed by function A to function B

Figure 12.5: Circular-Buffer Organization of Overlapped Windows

Assume that now the function E calls function F. This call cannot be serviced as the circular buffer already has allowed number of function call, unless you free space equivalent to exactly one window. This condition can easily be determined as current window pointer on incrementing will be equal to saved window pointer. Now, we need to create space; how can we do it? The simplest way will be to swap F_A register to memory and use that space. Thus, an N window register file can support N –1 level of function calls.

Thus, the register file, organized in the form as above, is a small fast register buffer that holds most of the variables that are likely to be used heavily. From this point of view the register file acts almost like a cache memory. So let us find how the two approaches are different.

Characteristics of large-register-file and cache organizations

Large Register File	Cache
Hold local variables for almost all functions. This saves time.	Recently used local variables are fetched from main memory for any further use. Dynamic use optimises memory.
The variables are individual.	The transfer from memory is block wise.
Global variables are assigned by the compilers.	It stores recently used variables. It cannot keep track of future use.
Save/restore needed only after the maximum call nesting is over (that is n – 1 open windows) .	Save/restore based on cache replacement algorithms.
It follows faster register addressing.	It is memory addressing.

The basic difference is due to addressing overhead of the two approaches. Small Register (R) address are smaller than the Cache reference, which are generated from a long memory address. Thus, for simple variables access register file is superior to cache memory. However, even in RISC computer, performance can be enhanced by the addition of instruction cache.

Check Your Progress 2

1. State True or False in the context of RISC architecture:

   a. RISC has a large register file so that more variables can be stored in register or longer periods of time.

   b. Only global variables are stored in registers.

   c. Variables are passed as parameters in registers using temporary registers in a window.

   d. Cache is superior to a large register file as it stores most recently used local scalars.

2. An overlapped register window RISC machine is having 32 registers. Suppose 8 of these registers are dedicated to global variables and the remaining 24 are split for incoming parameters, local and scalar variables and outgoing parameters. What are the ways of allocating these 24 registers in the three categories?

   .....

   .....

   .....

12.5 COMMENTS ON RISC

Let us now try and answer some of the comments that are asked for RISC architectures. Let us provide our suggestions on those.

CISCs provide better support for high-level languages as they include high-level language constructs such as CASE, CALL etc.

Yes CISC architecture tries to narrow the gap between assembly and High-Level Language (HLL); however, this support comes at a cost. If the architect provides a feature that looks like the HLL construct but runs slowly, or has many options, the compiler writer may omit the feature, or even, the HLL programmer may avoid the construct, as it is slow and cumbersome. Thus, the comment above does not hold.

It is more difficult to write a compiler for a RISC than a CISC.

The studies have shown that it is not so due to the following reasons:

If an instruction can be executed in more ways than one, then more cases must be considered. For it the compiler writer needed to balance the speed of the compilers to get good code. In CISCs compilers need to analyze the potential usage of all available instruction, which is time consuming. Thus, it is recommended that there is at least one good way to do something. In RISC, there are few choices; for example, if an operand is in memory, it must first be loaded into a register. Thus, RISC requires simple case analysis, which means a simple compiler, although more machine instructions will be generated in each case.

RISC is tailored for C language and will not work well with other high level languages.

But the studies of other high-level languages found that the most frequently executed operations in other languages are also the same as simple HLL constructs found in C, for which RISC has been optimized. Unless a HLL changes the paradigm of programming you may get similar result.

The good performance is due to the overlapped register windows; the reduced instruction set has nothing to do with it.

Certainly, a major portion of the speed is due to the overlapped register windows of the RISC that provide support for function calls. However, please note this register windows are possible due to reduction in control unit size. In addition, the control is simple in RISC than CISC, thus further helping the simple instructions to execute faster.

12.6 RISC PIPELINING

Instruction pipelining is often used to enhance performance. A RISC machine, in general, consists of two types of instructions:

a) The memory reference instructions, which are load and store instructions (Figure 12.1(b)). These instructions are used to bring/send data in registers from/to memory.

b) Data processing instructions (ADD instruction in Figure 12.1(b)), which perform the operation on register operands.

A memory reference instruction may be divided into the following pipeline stages:

FI: Fetch the Instruction from a memory address.
EI: Compute the effective address of the operand in the memory using the addressing modes. This may be similar to execution of an instruction.
TD: Transfer data from/to register to/from memory location

The data processing instruction would just require two pipeline stages:
FI: Fetch the Instruction from a memory address.

EI: Execute the instruction on register operands, result is stored in register.

Let us explain pipelining in RISC with an example program that uses instruction set given in Figure 12.1(b), with few additional instruction MUL, which use the same format as ADD instruction. The program segment implements the following expression:

$Z = (A + B) \times C$

(1) LDA R1, A (Load memory location A to R1)
(2) LDA R2, B (Load memory location B to R2)
(3) ADD R1, R2 (R1 ← R1 + R2)
(4) LDA R2, C (Load memory location C to R2)
(5) MUL R1, R2 (R1 ← R1 × R2)
(6) STR R1, Z (Store result in in memory location Z)

As discussed earlier, each of the instructions (1), (2), (4) and (6) will be processed in three stages and each of the instructions (3) and (5) will be processed in two stages. Assuming that one stage is executed in one clock cycle, a total of 4×3+2×2=16 clock cycles would be required if these instructions are not executed without using an instruction pipeline. However, if a pipeline that allows various overlapping stages to execute in parallel would result in execution of these instructions in only 8 clock cycles (Please refer to Figure 12.6)

(1)	LDA R1, A	FI	EI	TD					
(2)	LDA R2, B		FI	EI	TD				
(3)	ADD R1, R2			FI		EI			
(4)	LDA R2, C				FI	EI	TD		
(5)	MUL R1, R2					FI		EI	
(6)	STR R1, Z						FI	EI	TD
	Clock Cycles	1	2	3	4	5	6	7	8
	Time = 8 units								

Figure 12.6: RISC Pipelining

You may please note that the pipeline as shown above suffers from data dependencies at instruction (3) and (5). In both these cases, the preceding data access instruction must complete to allow the execution of these instructions. In addition, an instruction pipeline may suffer due to presence of branch instruction penalties. Next, we discuss about how such problems can be minimized

Optimization of Pipelining

RISC machines can employ a very efficient pipeline scheme because of the simple and regular instructions. Like all other instruction pipelines RISC pipeline suffer from the problems of data dependencies and branching instructions. The data dependency problem can be handled using an optimizing compiler, which can reschedule some of the instructions. For example, in the given program segment the, following changes may minimize the data dependency:

Interchange instruction (3) and instruction (4), in addition, instead of loading memory location C in R2 use R3 register. Accordingly, in instruction (5) change R2 to R3. The new instruction pipeline is shown in Figure 12.7.

(1)	LDA R1, A	FI	EI	TD					
(2)	LDA R2, B		FI	EI	TD				
(4)	*LDA **R3**, C*			FI	EI	TD			
(3)	*ADD R1, R2*				FI	EI			
(5)	MUL R1, **R3**					FI	EI		
(6)	STR R1, Z						FI	EI	TD
	Clock Cycles	1	2	3	4	5	6	7	8
	Time = 8 units								

Figure 12.7: Pipeline without data dependencies

The second problem of Branch instruction penalty can be optimised in RISC by using several techniques. For example, consider that a conditional branch instruction: "In case after the computation, the value R1 register is zero, then instruction 6 is not executed and the program jumps to instruction 7" exists after instruction 5. This modified instruction sequence is shown in Figure 12.8.

(1)	LDA R1, A	FI	EI	TD						
(2)	LDA R2, B		FI	EI	TD					
(4)	*LDA **R3**, C*			FI	EI	TD				
(3)	*ADD R1, R2*				FI	EI				
(5.a)	MUL R1, **R3**					FI	EI			
(5.b)	If R1=0 JMP to (7)						FI	EI		
(6)	STR R1, Z							FI	EI	TD
(7)	ADD R2, R3								FI	EI
	Clock Cycles	1	2	3	4	5	6	7	8	9

Figure 12.8: Pipeline with Brach Instruction

The problem with this instruction cycle is that the instruction 6 has already been fetched to the pipeline, therefore, in case R1 has zero value the pipeline should be emptied, i.e., instruction 6 will be removed from the pipeline. After that the instruction (7) should be stated again. There are two possible solutions to this problem. First the fetch of instruction (6) may be delayed for a cycle, so that the decision, whether the branch is to be taken or not would be made. Based on that the next instruction would be fetched. This is shown in Figure 12.9.

(1)	LDA R1, A	FI	EI	TD							
(2)	LDA R2, B		FI	EI	TD						
(4)	LDA R3, C			FI	EI	TD					
(3)	ADD R1, R2				FI	EI					
(5.a)	MUL R1, R3					FI	EI				
(5.b)	If R1=0 JMP to (7)						FI	EI			
(5.c)	*DO NOTHING instruction*							FI	EI		
(6) or (7)	STR R1, Z ADD R2, R3								FI FI	EI EI	TD
	Clock Cycles	1	2	3	4	5	6	7	8	9	10

Figure 12.9: Pipeline with reduced Brach penalty

Another way to handle the branch penalty is by moving the branch instruction, so that the branch decision is known prior to the execution of next instruction after the branch instruction. Figure 12.10 shows this solution.

(1)	LDA R1, A	FI	EI	TD						
(2)	LDA R2, B		FI	EI	TD					
(4)	LDA R3, C			FI	EI	TD				
(3)	ADD R1, R2				FI	EI				
(5.b)	*If R1=0 or R3 = 0 JMP to (7)*					FI	EI			
(5.a)	*MUL R1, R3*						FI	EI		
(6) or (7)	STR R1, Z ADD R2, R3							FI FI	EI EI	TD
	Clock Cycles	1	2	3	4	5	6	7	8	9

Figure 12.10: Pipeline with optimised Brach penalty

Please note that the instruction at (5.b) shows a hypothetical instruction that checks two conditions at a time. The purpose here is to demonstrate the concept, therefore, such instruction has been shown. Please also note the change in the sequence of instructions (5.a) and (5.b). Please also note that decision to take the branch has been taken at clock cycle 6, therefore, at lock cycle 7, it will be known, which of the two instructions (6) or (7) is to be fetched.

Finally, let us summarize the basic differences between CISC and RISC architecture. The following table lists these differences:

CISC	RISC
1. In general, large number of instructions	1. Relatively fewer instructions than CISC
2. Employs a variety of data types and a large number of addressing modes.	2. Relatively fewer addressing modes.
3. Variable-length instruction formats.	3. Fixed-length instructions, easy to decode instruction format.
4. Instructions manipulate operands residing in memory.	4. Mostly register-register operations. The only memory access is through explicit load and store instructions.
5. Number of Cycles Per Instruction varies from 1-20 depending upon the instruction.	5. Number of cycles per instruction is one as it uses pipelining. Pipeline in RISC is optimised because of simple instructions and instruction formats.
6. About 32 general purpose register, but no support is available for the parameter passing and function calls.	6. Large number of registers, which are used as Global registers and as a register based procedural call and parameter passing.
7. Microprogrammed Control Unit.	7. Hardwired Control Unit.

Check Your Progress 3

1. What are the problems, which prevent RISC pipelining to achieve maximum speed?

   .....

   .....

   .....

2. How can the above problems be handled?

.....

.....

.....

3. What are the problems of RISC architecture? How are these problems compensated such that there is no reduction in performance?

.....

.....

.....

12.7 SUMMARY

RISC represents new styles of computers that take less time to build yet provide a higher performance. While traditional machines support HLLs with instruction that look like HLL constructs, this machine supports the use of HLLs with instructions that HLL compilers can use efficiently. The loss of complexity has not reduced RISC's functionality; the chosen subset, especially when combined with the register window scheme, emulates more complex machines. Thus, we see that because of all the features discussed above, the RISC architecture should prove to better for certain applications.

In this unit we have also covered the details of the pipelined features of the RISC architecture, which support this architecture to show better performance.

12.8 SOLUTIONS/ ANSWERS

Check Your Progress 1

1.

- Speed of memory is slower than the speed of CPU.
- Microcode implementation is cost effective and easy.
- The intention of reducing code size.
- For providing support for high-level language.

2.

a) False
b) False
c) False

Check Your Progress 2

1.

(a) True
(b) False
(c) True
(d) False

2. Assume that the number of incoming parameters is equal to the number of outgoing parameters.

   Therefore, Number of locals = 24 –(2 × Number of incoming parameters)

   Return address is also counted as a parameter, therefore, number of incoming parameters is more than or equal to 1 or in other terms the possible combination, are:

C.loc

Incoming Parameter Registers	Outgoing Parameter Registers	No. of Local Registers
1	1	22
2	2	20
3	3	18
4	4	16
5	5	14
6	6	12
7	7	10
8	8	8
9	9	6
10	10	4
11	11	2
12	12	0

Check Your Progress 3

1. The following are the problems:
   - Branch instruction
   - The data dependencies between the instructions

2. It can be improved by:
   - Changing the sequence of some instruciton
   - causing optimized/ delayed jumps/loads etc.

3. The problems of RISC architecture are:
   - More instructions to achieve the same amount of work as CISC.
   - Higher instruction traffic
   - However, the cycle time of one instruction per cycle and instruction cache in the chip may compensate for these problems.